Supplementary Information

A flexible modelling framework for estimating thermal sensitivity across life

Daniel W.A. Noble*[1], Pieter A. Arnold[1], Patrice Pottier[2]

[1] *Division of Ecology and Evolution, Research School of Biology, Australian National University, Canberra, Australia*

[2] *Department of Biological and Environmental Sciences, University of Gothenburg, Gothenburg, Sweden*

2026-05-12

Table of contents

*Corresponding author: daniel.noble@anu.edu.au

1 Introduction

This supplement walks through the joint Bayesian 4-parameter logistic (4PL) framework described in the main manuscript. We provide a set of functions that allow you to apply the full workflow allowing the 4PL models to be fit and TLS parameters (z, $CT_{max_{1hr}}$, T_{crit}) to be extracted and also providing opportunities to integrate these TLS parameters to predict Heat Injury (HI) and survival fraction. These can also be combined with a repair model to allow damage (and therefore reductions in survival) to be repaired through a temperature time series. The functions are located in the `R/` directory of the project repository — there is no package to install. Sourcing the files at the top of an analysis (see below) makes every function available to call.

The functions are designed to be integrated together as a single pipeline:

1. **Standardise** raw experimental data to a common column schema (`temp`, `duration`, `n_total`, `n_surv`, …). This function ensures that the naming of columns is consistent to make it easier for downstream processing of models.
2. **Fit** the joint Bayesian 4PL with the built-in `beta_binomial(link = "identity")` family and asymptote-bounded reparameterisation.
3. **Extract** the classical TDT quantities — thermal sensitivity (z), critical thermal limit at one hour ($CT_{max_{1hr}}$), and the critical-injury threshold temperature (T_{crit}) — all with full posterior uncertainty.
4. **Predict** survival under fluctuating field temperature traces, including heat-injury accumulation with optional Sharpe-Schoolfield repair.
5. **Plot** each step with shared theming so figures across the workflow read consistently.

The remainder of the supplement walks the pipeline through a controlled simulation where the truth is known (§ A Tutorial with Simulations), then applies the same machinery end-to-end to a real lethal-TDT dataset on brown shrimp (§ Case Study 1: Shrimp data).

1.1 Loading the function library

The supplement assumes the project repository is the current working directory, so `here::here()` can resolve paths to `R/`. The chunk below loads required packages and sources every file in the function library.

```
# Source the project function library (12 thematic files under R/).
tdt_lib_files <- c(
  "utils.R", "standardize_data.R", "priors.R", "fit_4pl.R",
  "predict_survival_curves.R", "extract_tdt.R", "tdt_landscape.R",
  "repair.R", "temperature_scenarios.R", "predict_heat_injury.R",
  "plotting.R", "diagnostics.R"
)
for (f in tdt_lib_files) source(here::here("R", f))
```

1.2 What each function does

Table S1 lists every exported function in the library, grouped by pipeline stage. Each function is single-purpose; the full roxygen2 documentation (with runnable `@examples`) lives in the corresponding R file.

1.3 Testing

Every function has matching unit tests under tests/testthat/. Fast tests (data manipulation, formula construction, prior structure, plus the pure-math 4PL inversion and planted-dose math) run in seconds via `testthat::test_dir("tests/testthat")`. A small set of brms-fitting integration tests is gated behind `RUN_BRMS_TESTS=true`: those build a small cached fixture fit and assert that `extract_tdt()` and `predict_heat_injury()` recover known simulation truths within their credible intervals.

2 A Tutorial with Simulations

This section is a guided tutorial using simulated data. We show, step by step, how the Bayesian hierarchical 4PL — under both binomial sampling and the more realistic beta-binomial (overdispersed) case – can recover thermal sensitivity (z) and $CT_{max_{1hr}}$ from the single joint model, yet provide substantial downstream benefits in terms of propagating uncertainty and giving more flexibility. We then complement the simulation with a few case studies which walk through real data that is presented within the main manuscript in other sections.

2.0.1 *Simulating Data*

2.0.1.1 Step 1 — Setting up parameters

We choose values that might be typical of a thermal mortality assay. The four 4PL parameters are the lower and upper asymptotes (ℓ, u), the slope on the $\log_{10}$-time axis (k), and a midpoint that varies linearly with temperature: $\text{mid}(T) = \beta_0 + \beta_1(T - \bar{T})$. Each value below has a clear biological meaning.

```
true_params <- list(
  ell      = 0.03,   # 3% of individuals survive even at the longest exposures
  u        = 0.97,   # 97% are alive at very short exposures
  k        = 8,      # steepness of the survival drop on the log10(t) axis
  m_beta0  = 1.5,    # midpoint at the grand-mean temperature: ~31.6 min
  m_beta1  = -0.15,  # midpoint drops 0.15 log10-min per °C; z = -1/beta1   6.7 °C
  T_bar    = 34      # grand-mean temperature
)
```

The implied thermal sensitivity is $z = -1/\beta_1 \approx 6.7°\text{C}$ — the temperature change that shifts LT50 by a factor of 10. From the same parameters we can also compute the simulation truth for the uncentred TDT intercept $\alpha = \beta_0 + \frac{1}{k}\log\frac{u-0.5}{0.5-\ell} - \beta_1\bar{T}$ and the critical thermal limit at 1

Table S1: Functions exported by the TDT analysis library, grouped by pipeline stage. Each function is documented with roxygen2 in its R file; runnable examples accompany every function.

Stage	Function	Purpose
Data	standardize_data()	Rename raw columns to project schema; attach metada
Model spec	make_4pl_priors()	Weakly informative priors for the disjoint-bounds 4PL.
Model spec	make_4pl_formula()	brms non-linear formula; identity link, all four sub-para
Fit	fit_4pl()	Joint Bayesian 4PL fit; returns a tdt_4pl_workflow ob
Diagnostics	diagnose_tdt_fit()	Rhat / ESS / divergences / treedepth, with pass-fail fl
Diagnostics	tdt_parameter_table()	Posterior summary on the natural scale (low, up, k, mi
Predictions	predict_survival_curves()	Survival curves on a temp x duration grid with credibl
Predictions	derive_tdt_landscape()	Dense 2-D survival surface for a heatmap.
TDT quantities	derive_ltx_curve()	Time-to-x-survival across temperature (per-draw invers
TDT quantities	derive_temperature_for_duration()	Temperature at survival = X for a fixed exposure dura
TDT quantities	derive_tdt_parameters()	Per-draw log-linear regression of LT_x on temperature
TDT quantities	extract_tdt()	Bundled wrapper: z, CTmax, T_crit posteriors in one
Heat injury	make_temperature_scenarios()	Three reference traces (flat / single spike / multi-spike
Heat injury	planted_dose_from_trace()	Analytical HI under known z, CTmax, T_c (validation
Heat injury	predict_heat_injury()	Posterior HI and survival under a temperature trace.
Heat injury	repair_rate_schoolfield()	Sharpe-Schoolfield repair TPC (Kelvin internally).
Plotting	plot_survival_curves()	Posterior curves with observed proportion overlay.
Plotting	plot_ltx_curve()	LT_x curve on a log-time axis (classical TDT view).
Plotting	plot_tdt_landscape()	Survival heatmap with contour overlay.
Plotting	plot_temperature_density()	Posterior density for CTmax / T_crit with a 95-percen
Plotting	plot_temperature_scenarios()	Three traces stacked vertically.
Plotting	plot_heat_injury()	HI and predicted survival trajectories with CrI bands.
Plotting	plot_repair_rate()	Sharpe-Schoolfield TPC curve.

hour $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — both approaches below will be checked against these known values.

Recall that we can calculate $ $, z and $CT_{max_{1hr}}$ from the true parameters using the equations in the main manuscript. We'll do that now so we have a comparator for later.

```
true_z      <- -1 / true_params$m_beta1
true_alpha  <- true_params$m_beta0 +
  (1 / true_params$k) *
  log((true_params$u - 0.5) / (0.5 - true_params$ell)) -
  true_params$m_beta1 * true_params$T_bar
true_CTmax  <- (log10(60) - true_alpha) / true_params$m_beta1

# T_crit under the rate-multiplier definition (see §[Deriving z, CTmax, and
# T_crit from the posterior]). For each posterior draw, T_crit is computed as
# CTmax_1hr + z * log10(r*/100) where r* is sampled uniformly on log10
# across the default range c(0.1, 1) % HI per hour. The MEDIAN of that
# distribution lies at log10(r*/100) = -2.5 (the geometric mean of the
# range), so:
true_T_crit <- true_CTmax - 2.5 * true_z
```

2.0.1.2 Step 2 — Simulated study design

We mimic a textbook TDT experiment: a grid of five assay temperatures crossed with six log-spaced exposure durations, repeated 30 times per cell, with 30 individuals tested per replicate. That gives 900 trials and 27,000 individuals — large enough to recover the parameters with negligible uncertainty, but small enough to fit in a few minutes. Of course, in practice we will not have this much data as will be seen in the case studies, but simulations are a good starting point to understand the models being fit and the conversions.

```
temps     <- c(30, 32, 34, 36, 38)              # 5 assay temperatures (°C)
durations <- c(1, 5, 15, 45, 135, 405)          # 6 durations (minutes, log-spaced)
n_rep     <- 30                                 # 30 replicates per cell
n_per_rep <- 30                                 # 30 individuals per replicate

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,                      # mean-centred T
    mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,  # mid(T)
    p_true  = true_params$ell +
      (true_params$u - true_params$ell) /
      (1 + exp(true_params$k * (log10_t - mid))),
```

```
    n = n_per_rep
  )
```

2.0.1.3 Step 3 — Look at the truth before simulating

Before generating noisy data, let's plot the *true* survival surface (Figure S1) so we know what the model is being asked to recover. Each line is the 4PL at one assay temperature, plotted against $\log_{10}$ exposure time.

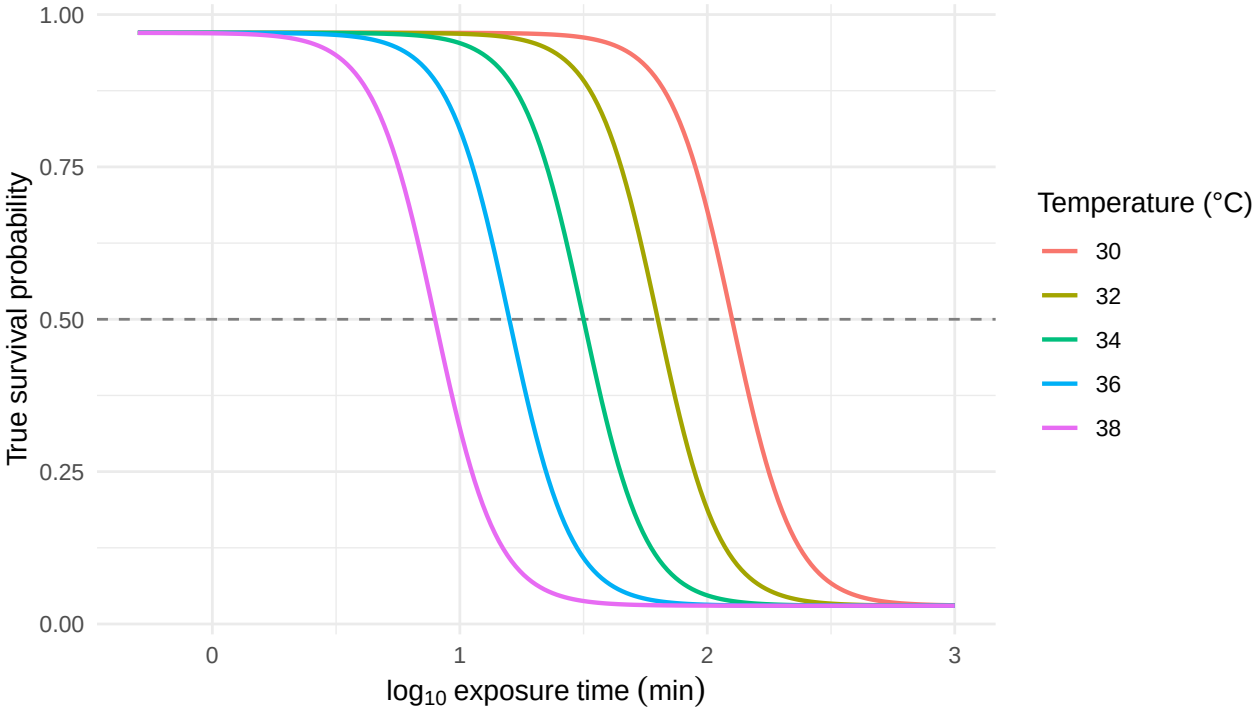

Figure S1: The known 4PL dose-response surface used to generate the simulated data, evaluated on a dense time grid at each of the five assay temperatures. The horizontal line marks 50% survival; where each curve crosses it gives the true LT50 at that temperature.

Notice the curves shift left as temperature rises: hotter temperatures kill faster, so the LT50 is reached at shorter durations. That horizontal shift in midpoints is exactly what β_1 encodes.

2.0.1.4 Step 4 — Generate noisy data: binomial and beta-binomial

Real survival counts have at least two flavours of noise. **Binomial** sampling is the textbook case: each replicate's survival count is a draw from Binomial(n, p) with p from the 4PL surface. **Beta-binomial** sampling adds *overdispersion* — replicate-to-replicate variability in p that the binomial cannot capture, e.g., from cohort effects or unmodelled heterogeneity. Real TDT data almost always show overdispersion so in many cases the beta binomial will be a better fit.

```r
# (a) Binomial: y ~ Binomial(n, p_true)
sim_bin <- design |>
  dplyr::mutate(y = rbinom(dplyr::n(), size = n, prob = p_true))

# (b) Beta-binomial: replicate-level p drawn from Beta(p*phi, (1-p)*phi)
phi <- 5  # smaller phi = more overdispersion
sim_bb <- design |>
  dplyr::mutate(
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

2.0.2 *The classical two-stage TDT pipeline*

We first run the simulated data through the **classical two-stage TDT pipeline** (Ørsted *et al.* 2022; Rezende *et al.* 2014), which is dominant in the thermal-tolerance literature.

1. **Stage 1.** At each assay temperature, fit a logistic dose-response curve to the count data and read off a point estimate of $\log_{10} \mathrm{LT50}_T$.
2. **Stage 2.** Regress those Stage-1 point estimates on assay temperature with ordinary least squares, then derive $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ from the fitted line.

2.0.2.1 Stage 1 — Dose-response curves at each temperature

We fit a binomial GLM with a logit link separately at each assay temperature. This is the most-common Stage-1 specification in the TDT literature: a two-parameter logistic with asymptotes constrained to 0% and 100%. The midpoint $\log_{10} \mathrm{LT50}_T = -\beta_0/\beta_1$ falls out of the GLM's two coefficients.

```r
fit_stage1 <- function(sim_data) {
  sim_data |>
    dplyr::group_by(T) |>
    dplyr::group_modify(function(d, key) {
      f  <- glm(cbind(y, n - y) ~ log10_t, data = d, family = binomial())
      co <- coef(f)
      tibble::tibble(log10_lt50 = as.numeric(-co[1] / co[2]))
    }) |>
    dplyr::ungroup()
}

stage1_bin <- fit_stage1(sim_bin)
stage1_bb  <- fit_stage1(sim_bb)
```

With the simulation's true asymptotes near 0 and 1, the 2PL's forced-asymptote constraint introduces negligible bias here, so each Stage-1 estimate sits close to the simulation truth.

Table S2: Per-temperature Stage-1 estimates of $\log_{10}$ LT50 from a binomial GLM with logit link, fit separately at each of the five assay temperatures. Truth values are the simulated midpoints (which coincide with $\log_{10}$ LT50 here because the simulation asymptotes are near 0 and 1).

T (°C)	Truth log10(LT50)	Binomial log10(LT50)	Beta-binomial log10(LT50)
30	2.1	2.061	2.082
32	1.8	1.795	1.8
34	1.5	1.48	1.528
36	1.2	1.202	1.195
38	0.9	0.907	0.954

2.0.2.2 Stage 2 — log-linear TDT regression

Stage 2 regresses Stage-1 $\log_{10}$ LT50 on assay temperature with ordinary least squares. The classical TDT quantities — $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — fall out of the fitted line.

```
fit_stage2 <- function(stage1) {
  fit2 <- lm(log10_lt50 ~ T, data = stage1)
  co   <- coef(fit2)

  tibble::tibble(
    quantity = c("z (°C)", "CTmax at 1 hr (°C)"),
    est      = c(as.numeric(-1 / co["T"]),
                 as.numeric((log10(60) - co["(Intercept)"]) / co["T"]))
  )
}

ts_bin <- fit_stage2(stage1_bin)
ts_bb  <- fit_stage2(stage1_bb)
```

Figure S2 visualises the Stage-2 fit. Each point is one Stage-1 $\log_{10}$ LT50 estimate from Table S2; the solid line is the OLS regression through them. The slope is β_1, from which $z = -1/\beta_1$ — i.e., the temperature increase that causes LT50 to change by a factor of 10. The line's height at $\log_{10} 60 \approx 1.78$ (1 hour) is at the temperature equal to $CT_{max_{1hr}}$ — the vertical dotted line drops from this intersection straight to the temperature axis to mark it. The dashed grey line is the simulation truth.

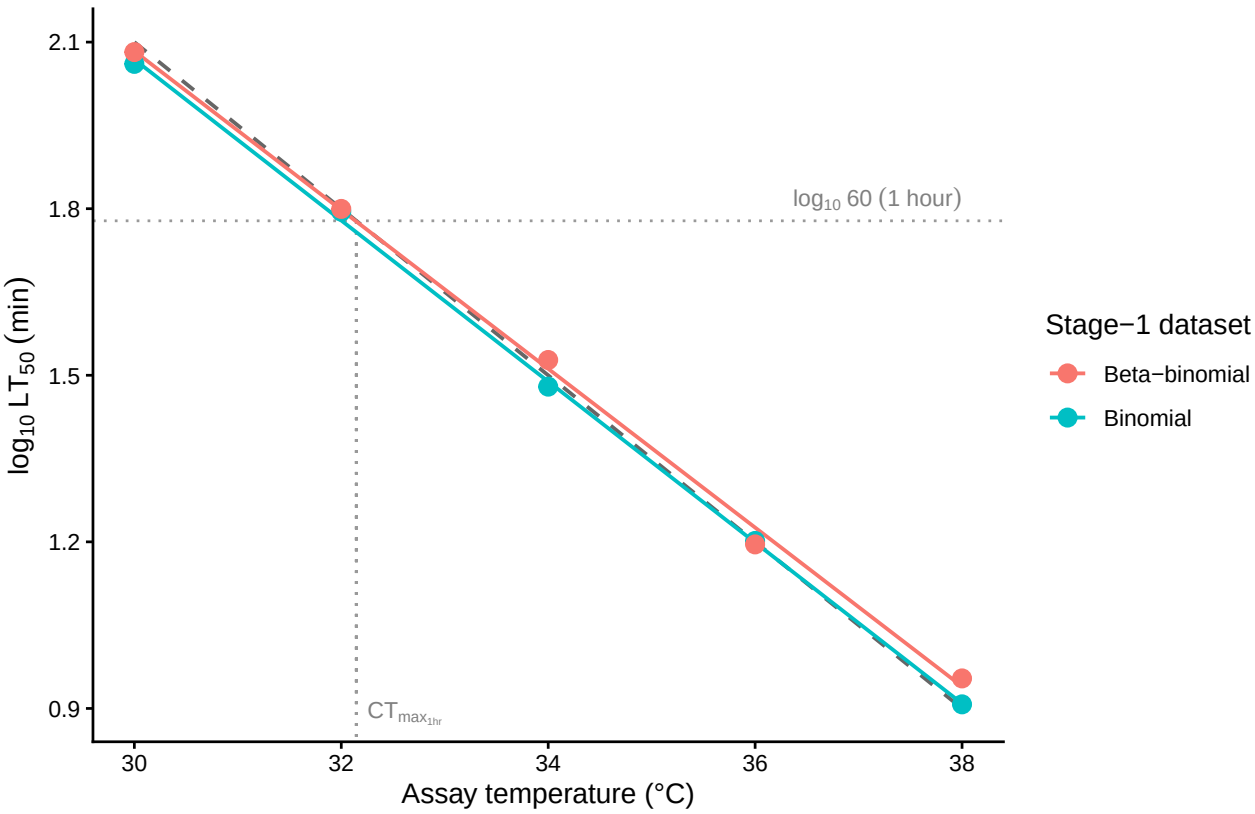

Figure S2: Stage-2 OLS regression of $\log_{10}$ LT50 on assay temperature, fit separately to the binomial and beta-binomial simulated datasets. Points are the Stage-1 per-temperature LT50 estimates; solid coloured lines are the OLS fits; the dashed grey line is the simulation truth. The slope of each line equals β_1, from which $z = -1/\beta_1$. The horizontal dotted line marks $\log_{10} 60$ (1 hour); the vertical dotted line drops from where the truth crosses 1 hour straight down to the temperature axis, marking $CT_{max_{1hr}}$ visually.

This is the entire two-stage workflow. The point estimates that come out — z and $CT_{max_{1hr}}$ — are what the field reports. We carry them forward (`ts_bin` and `ts_bb`) for the side-by-side comparison once we have the joint Bayesian results.

> **!** Important
>
> Two-stage steps often ignore uncertainty in estimates in both fitting and error propagation. Uncertainty in estimates can be propagated using the SE's and the delta method, but only at each individual stage.

2.0.3 *A joint Bayesian 4PL*

Instead of fitting a separate logistic at each assay temperature and then regressing the LT50s, the joint fit estimates every parameter — ℓ, u, k, β_0 and β_1 — in a single hierarchical model on the raw counts. We now use the same simulated data to derive the z and $CT_{max_{1hr}}$ from the posterior, in one model rather than two.

2.0.3.1 Specifying the 4PL in `brms`

The model has two pieces. The first is the 4PL itself — survival probability as a function of $\log_{10}$ exposure time:

$$p_{ij} \;=\; \ell \;+\; \frac{u-\ell}{1+\exp\!\big(k\,(\log_{10} t_{ij} - \mathrm{mid}_{ij})\big)}, \tag{S1}$$

where ℓ and u are the lower and upper asymptotes, k is the slope on the $\log_{10} t$ axis, and mid_{ij} is the value of $\log_{10} t$ at which the curve crosses the midpoint between ℓ and u. The second piece lets the midpoint vary linearly with mean-centred temperature:

$$\mathrm{mid}_{ij} \;=\; \beta_0 + \beta_1\,(T_{ij} - \bar{T}), \tag{S2}$$

so that β_1 encodes how fast the dose-response curve shifts on the time axis as assay temperature changes; $\bar{T}$ is the grand-mean temperature, included so that β_0 is interpretable as the midpoint at the average assay temperature.

The function library exposes this model as a single call: `fit_4pl()` constructs the brms `bf()` formula, the default weakly-informative priors, and the sampling controls, and returns a workflow object containing the fit. Internally the formula uses the disjoint-bounds reparameterisation described in §Loading the function library — `low = 0.001 + \text{inv\_logit}(\text{lowraw}) \cdot 0.498` and `up = 0.501 + \text{inv\_logit}(\text{upraw}) \cdot 0.498` — so the 4PL output is guaranteed to lie in $(0, 1)$ regardless of where MCMC takes the unconstrained `lowraw`, `upraw`, and `logk` parameters. The default also puts a `temp_c` slope on every 4PL sub-parameter; when temperature has no real effect on a given parameter, the slope shrinks toward zero under the prior. We inspect the brms formula `fit_4pl()` builds with:

```
make_4pl_formula()
```

```
n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(upraw) * (
lowraw ~ temp_c
upraw ~ temp_c
logk ~ temp_c
mid ~ temp_c
```

> **!** Important
>
> The default model puts a `temp_c` slope on all four 4PL sub-parameters and includes no random effects. Random intercepts on `mid` can be added by passing `random_effects = c("Date", "Tank", ...)` to `fit_4pl()`. The simulation here has no batch structure so we leave it off; the shrimp case study below uses `random_effects = c("Date", "Tank")`.

2.0.3.2 Standardising and fitting

The first step is to pass the simulated data through `standardize_data()` so the columns match the schema the rest of the library expects (`temp`, `duration`, `logd`, `temp_c`, `n_total`, `n_surv`).

```
std_bin <- standardize_data(sim_bin,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
std_bb  <- standardize_data(sim_bb,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
```

We now call `fit_4pl()` twice — once with the built-in `binomial(link = "identity")` family for the non-overdispersed simulation, and once with the default `beta_binomial(link = "identity")` for the overdispersed case. Caching is handled by brms's `file = ...` mechanism, so subsequent renders reload the cached fits unless the formula or data changes.

```
wf_bin <- fit_4pl(std_bin,
                  family = binomial(link = "identity"),
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file = file.path(models_dir, "sim_4pl_binomial_v2"))

wf_bb  <- fit_4pl(std_bb,
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file = file.path(models_dir, "sim_4pl_betabinomial_v2"))

# Convenience: extract the brmsfit object for downstream brms helpers.
fit_bin <- wf_bin$fit
fit_bb  <- wf_bb$fit
```

We can also get an idea of how much variation our model explains that considers all sources of relevant variation.

```
# Obtain the Bayes R2 for each model
brms::bayes_R2(fit_bin)
```

```
    Estimate    Est.Error     Q2.5     Q97.5
R2 0.9875292 0.0001264011 0.987227 0.9877159
```

```
brms::bayes_R2(fit_bb)
```

```
    Estimate  Est.Error      Q2.5     Q97.5
R2 0.9263016 0.00105021 0.9238746 0.9280369
```

Table S3: Posterior medians and 95% credible intervals for the four 4PL parameters and the temperature slope, compared to the known truth used to generate the simulated data. Recovery is judged successful when the credible interval covers the truth.

Parameter	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
ell	0.03	0.0289	0.0243	0.0334	0.0242	0.0138	0.0346
u	0.97	0.9671	0.9629	0.9709	0.97	0.9619	0.977
k	8	8.3117	7.886	8.7684	7.3327	6.6161	8.1562
mid (intercept)	1.5	1.5013	1.4925	1.5097	1.5173	1.4966	1.5378
mid (slope)	-0.15	-0.1495	-0.1525	-0.1464	-0.1495	-0.1571	-0.142

2.0.3.3 Checking parameter recovery

We can now check how well our fitted model recovered the true parameters that generated the data. The most direct check is to compare posterior medians (with 95% credible intervals) to the known truth (Table S3). We have big sample sizes here so our values should be pretty close to our true values.

We can also overlay the posterior fitted curves on the simulated data (Figure S3). If the model has recovered the true surface, the fitted lines should track the simulated proportions at every assay temperature.

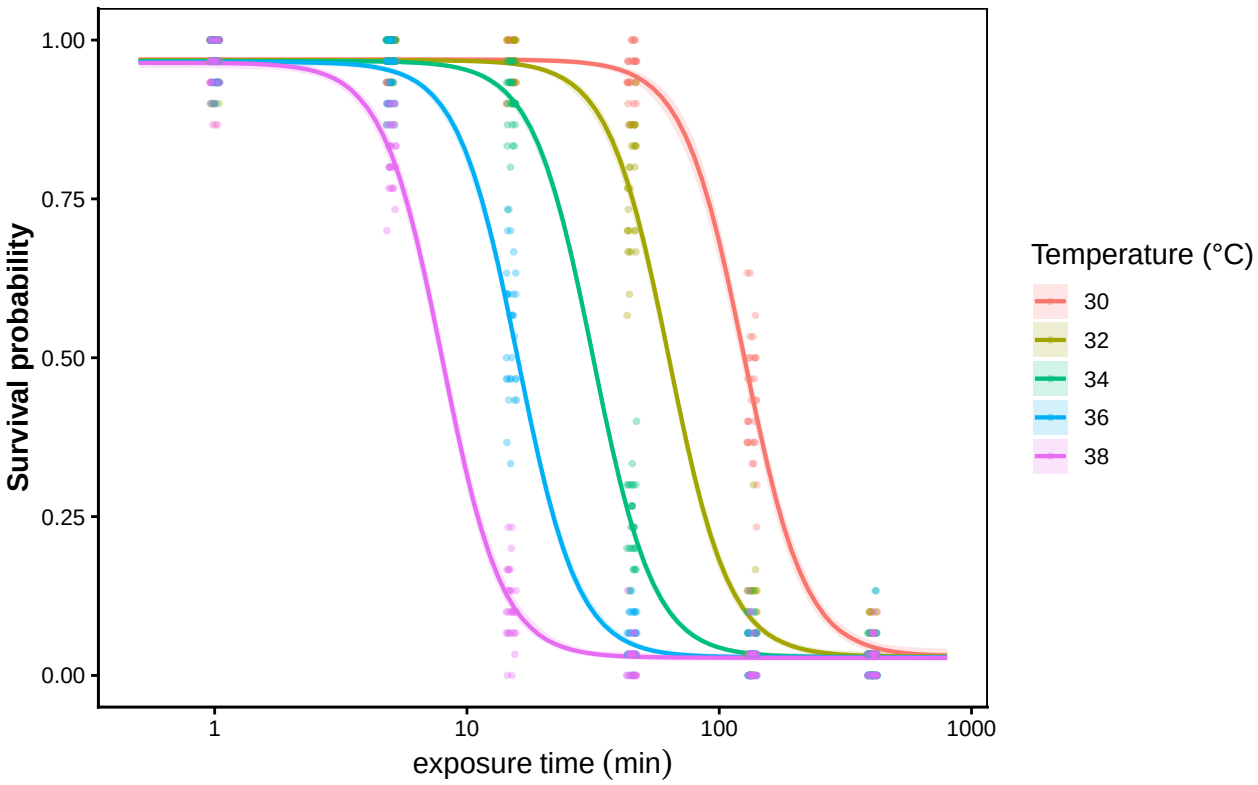

Figure S3: Posterior fitted 4PL curves (solid lines, with shaded 95% credible bands) overlaid on the simulated binomial data (points). One panel-coloured line per assay temperature. Where the fitted curves track the simulated proportions across all five temperatures, the model has recovered the underlying dose-response surface.

2.0.4 *Deriving z, $CT_{max_{1hr}}$, and T_{crit} from the posterior*

The whole point of fitting the joint Bayesian 4PL is that the classical TLS quantities fall out as transformations of the posterior we just fit — no second regression, no bootstrap. `extract_tdt()` bundles three of them:

- z — thermal sensitivity, derived by a per-draw log-linear regression of $\log_{10}$ LT50 on temperature (Equation 7 of the manuscript).
- $CT_{max_{1hr}}$ — temperature at 50% survival after 1 hour exposure, obtained by inverting the fitted 4PL at the reference time, per posterior draw.
- T_{crit} — the temperature below which damage accumulates more slowly than a chosen rate floor. The damage rate at temperature T in the TDT framework is $r(T) = 100 \cdot 10^{(T - CT_{max,1hr})/z}$ % LT50-dose per hour, so inverting for a chosen rate floor r^* gives

$$T_{crit}(r^*) = CT_{max,1hr} + z \cdot \log_{10}(r^*/100). \tag{S3}$$

Different r^* values give different T_{crit} thresholds, and the literature does not converge on a single value. Empirical breakpoint analyses across taxa as different as *Drosophila suzukii* and *Lemna gibba*

Table S4: Posterior medians and 95% credible intervals for thermal sensitivity (z), the critical thermal limit at a 1-hour reference duration ($CT_{max_{1hr}}$), and the rate-multiplier critical temperature (T_{crit}, integrated over $r^* \in [0.1, 1]$ % HI per hour). All three derived via `extract_tdt()` from a single posterior. Each row's truth value is computed analytically from `true_params`.

Quantity	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
z (°C)	6.67	6.69	6.55	6.82	6.72	6.42	7.08
CTmax at 1 hr (°C)	32.15	32.14	32.08	32.21	32.23	32.07	32.39
T_crit (°C)	15.48	15.5	12.27	18.61	15.4	12.1	18.69

fall in the range $r^* \in [0.1, 1]$ % HI per hour, corresponding to "$\sim$ 4 days to 1000 hours of continuous exposure to reach LT_{50}" [Jørgensen *et al.* (2021); Arnold *et al.* (2025); **havro_faber_2023**]. Rather than commit to one r^*, we treat it as an unknown with a uniform prior on $\log_{10} r^*$ over that range and integrate; the posterior of T_{crit} then carries both parameter uncertainty (in $CT_{max,1hr}$ and z) and operational uncertainty (in the choice of r^*). The median of that pooled posterior sits at $CT_{max,1hr} - 2.5 \cdot z$, which corresponds to $r^* \approx 0.3$ % HI per hour (the geometric mean of the rate range).

All three quantities inherit the same posterior, so every credible interval reflects joint uncertainty in $(\ell, u, k, \beta_0, \beta_1)$.

```
# Data are in minutes (standardize_data was called with duration_unit = "minutes"),
# so set time_multiplier = 1 to keep the model's time axis unchanged on output,
# and t_ref = 60 to read CTmax at the 1-hour reference. T_crit uses the
# rate-multiplier integration; default TC_rate_range = c(0.1, 1) % HI per hour.
et_bin <- extract_tdt(wf_bin, t_ref = 60, time_multiplier = 1, ndraws = 1000)
et_bb  <- extract_tdt(wf_bb,  t_ref = 60, time_multiplier = 1, ndraws = 1000)
```

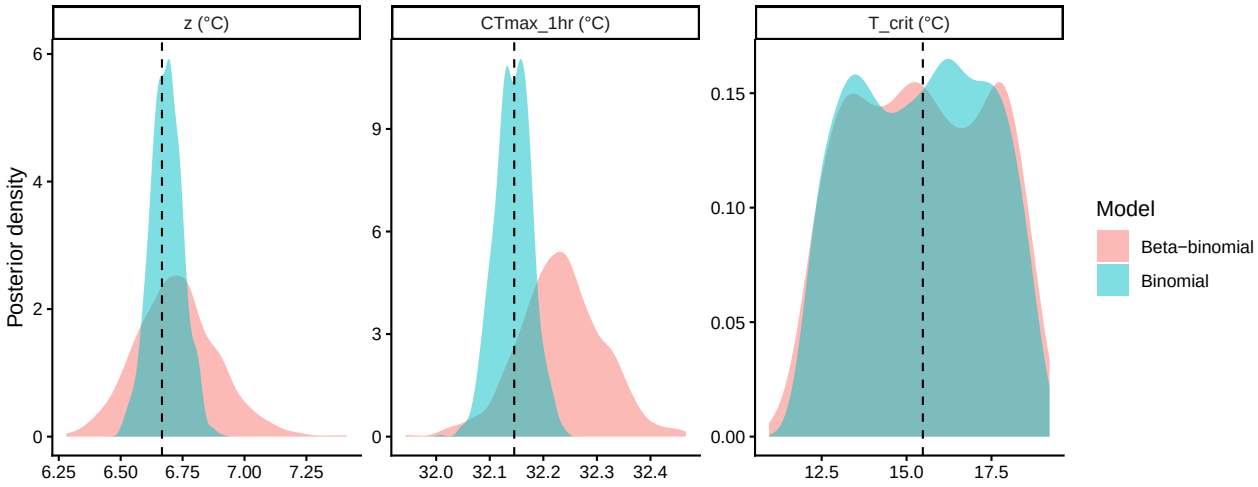

Figure S4: Posterior densities of z, $CT_{max_{1hr}}$, and T_{crit} from the binomial (blue) and beta-binomial (orange) joint fits. Vertical dashed lines mark the simulation truth. All three posteriors concentrate around their respective truths — illustrating that one fit, one library call (`extract_tdt()`), produces every classical TLS quantity with calibrated uncertainty and no extra regression step.

All three quantities recover their simulation truth in both fits (Table S4, Figure S4), with credible intervals reflecting joint uncertainty in all 4PL parameters. The key practical advantage of the joint approach over the two-step pipeline: every classical TLS quantity is one library call away from the fit we already have, and every quantity inherits the same calibrated uncertainty.

Posterior summary statistics — mean, SD, median, and 95% credible interval — are also readily available from the `$draws` slot of `extract_tdt()`'s output:

```
summarise_draws <- function(x) {
  q <- stats::quantile(x, c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
  tibble::tibble(mean = mean(x, na.rm = TRUE),
                 sd   = stats::sd(x, na.rm = TRUE),
                 median = q[2], lower = q[1], upper = q[3])
}

uncert_summary <- dplyr::bind_rows(
  summarise_draws(et_bin$z$draws$z)          |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(et_bin$CTmax$draws$temp)   |> dplyr::mutate(quantity = "CTmax at 1 hr (°C)",
  summarise_draws(et_bin$T_crit$draws$temp)  |> dplyr::mutate(quantity = "T_crit (°C)", likeli
  summarise_draws(et_bb$z$draws$z)           |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(et_bb$CTmax$draws$temp)    |> dplyr::mutate(quantity = "CTmax at 1 hr (°C)",
  summarise_draws(et_bb$T_crit$draws$temp)   |> dplyr::mutate(quantity = "T_crit (°C)", likeli
) |>
  dplyr::select(quantity, likelihood, mean, sd, median, lower, upper)

# Scalars for inline reporting in the next paragraph.
sd_z_bin  <- round(stats::sd(et_bin$z$draws$z),         2)
sd_z_bb   <- round(stats::sd(et_bb$z$draws$z),          2)
```

Table S5: Posterior summaries of z, $CT_{max_{1hr}}$, and T_{crit} from the joint Bayesian 4PL: posterior mean, standard deviation, median, and 95% credible interval, reported separately for the binomial and beta-binomial fits.

Quantity	Likelihood	Mean	SD	Median	Lower	Upper
z (°C)	Binomial	6.68	0.0657	6.69	6.55	6.82
CTmax at 1 hr (°C)	Binomial	32.14	0.0339	32.14	32.08	32.21
T_crit (°C)	Binomial	15.42	1.8996	15.5	12.27	18.61
z (°C)	Beta-binomial	6.73	0.1647	6.72	6.42	7.08
CTmax at 1 hr (°C)	Beta-binomial	32.23	0.0788	32.23	32.07	32.39
T_crit (°C)	Beta-binomial	15.42	1.9954	15.4	12.1	18.69

```
sd_ct_bin <- round(stats::sd(et_bin$CTmax$draws$temp),  2)
sd_ct_bb  <- round(stats::sd(et_bb$CTmax$draws$temp),   2)
ci_z_bin  <- round(diff(stats::quantile(et_bin$z$draws$z,          c(0.025, 0.975), names = FALS
ci_z_bb   <- round(diff(stats::quantile(et_bb$z$draws$z,           c(0.025, 0.975), names = FALS
ci_ct_bin <- round(diff(stats::quantile(et_bin$CTmax$draws$temp,  c(0.025, 0.975), names = FALS
ci_ct_bb  <- round(diff(stats::quantile(et_bb$CTmax$draws$temp,   c(0.025, 0.975), names = FALS

uncert_summary |>
  dplyr::rename(
    Quantity   = quantity,
    Likelihood = likelihood,
    Mean       = mean,
    SD         = sd,
    Median     = median,
    Lower      = lower,
    Upper      = upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

In short, both z and $CT_{max_{1hr}}$ are estimated with high precision and their credible intervals come straight from the joint posterior. If we fit this model using frequentist approaches we could do the same and use the delta-method, or bootstrapping to get uncertainty.

Under the binomial likelihood, the standard error (or SD of posterior) for z is 0.07 °C with a 95% credible interval 0.26 °C wide; $CT_{max_{1hr}}$ has SD 0.03 °C and a 95% credible interval width of 0.13 °C.

The beta-binomial fit is slightly wider on both quantities (SD on z: 0.16 °C; on $CT_{max_{1hr}}$: 0.08 °C) because ϕ has absorbed the extra-binomial variance and that uncertainty propagates honestly into the derived quantities — none of which falls out of the two-stage pipeline without an additional bootstrap or delta-method step.

Table S6: Point estimates (two-stage) and posterior medians (joint Bayesian) for thermal sensitivity (z) and $CT_{max_{1hr}}$, recovered by each of four estimator–dataset combinations from the same simulated datasets. Truth values are the simulation parameters.

Quantity	Truth	Two-stage (binomial)	Joint Bayesian (binomial)	Two-stage (beta-binomial)
z (°C)	6.67	6.9	6.69	6.99
CTmax at 1 hr (°C)	32.15	32	32.14	32.14

> **!** Important
>
> It is always important to evaluate whether a binomial and a beta-binomial model better fits real data. The extra dispersion, typical of real biological data, should be estimated when it's present as it has important consequences for inferences.

2.0.5 *Comparing the two pipelines*

2.0.5.1 Point-estimate agreement on the same data

Table S6 puts the two-stage point estimates next to the joint Bayesian posterior medians for both simulated datasets.

All four estimators land on essentially the same point estimates for z and $CT_{max_{1hr}}$, though we notice the two stage is biased upwards slightly, and all sit close to the simulation truth. The joint Bayesian 4PL gives the same answer the classical pipeline gives — it is *not* in disagreement with the field's existing point estimates, it reproduces them.

2.0.6 *Extending the model: temperature-varying shape parameters*

The closed-form derivations of z and $CT_{max_{1hr}}$ assumed that ℓ, u and k are scalars — they do not depend on T, so the asymmetry correction $\frac{1}{k}\log\frac{u-0.5}{0.5-\ell}$ is a single number rather than a function of temperature. This is convenient but not required. If ℓ, u and k also vary with temperature, then we need to predict how each of these parameters changes with temperature and apply a temperature-varying correction factor. Linear effects of temperature on the shape parameters of the dose-response curve are mechanistically interpretable and biologically plausible. Three concrete examples come to mind:

- `u ~ T_c` lets the upper asymptote (baseline survival at very short exposures) decline with rising assay temperature. This may be quite a sensible assumption if brief exposures at high T cause some immediate mortality.
- `ell ~ T_c` lets the residual surviving fraction at very long exposures depend on T. While this seems less plausible, if there is individual variation (which seems likely) some fraction may still survive, partiuclarly at less extreme temperatures, which could shift the lower baseline across each temperature treatment.

- `k ~ T_c` lets the dose-response steepness depend on T. Biologically this is the most interesting case: protein denaturation kinetics often look more threshold-like (sharper k) near a protein's melting temperature, so the slope at extreme assay temperatures may be steeper than at moderate ones and there is some suggestions that this could be true (Jørgensen *et al.* 2021; Kingsolver & Umbanhowar 2018).

When any shape parameter depends on T the asymmetry correction itself becomes a function of T:

$$\log_{10} \mathrm{LT50}(T) \;=\; \beta_0 + \beta_1 \,(T - \bar{T}) \;+\; \tfrac{1}{k(T)} \log \tfrac{u(T)-0.5}{0.5-\ell(T)}. \tag{S4}$$

The midpoint piece $\beta_0+\beta_1(T-\bar{T})$ is still a straight line in T, but the asymmetry-correction piece now varies with T through $k(T), u(T), \ell(T)$. Their sum is therefore non-linear in T. Two consequences follow, and they affect z and $CT_{max_{1hr}}$ differently: z becomes a *function* of temperature (no longer a single slope but a local value at each T). In contrast, $CT_{max_{1hr}}$ remains a *single* number but its value just shifts because the underlying curve is now bent rather than straight.

- **z becomes a function of temperature.** Since z is defined by the local rate of change of $\log_{10}\mathrm{LT50}$ with T, $z(T) = -1/\frac{d\log_{10}\mathrm{LT50}(T)}{dT}$. In other words, the local slope of $\log_{10}\mathrm{LT50}$ at any given T varies across the assay range because the shape of the dose-response curve changes with T. This curvature leads to the infinitesimal rates (local slopes) changing which affects the overall sensitivity, z. In these cases, it's better to estimate the local slope first. An overall z can then be obtained by averaging the local slopes if a single summary is needed (pooling the posteriors).
- **CT_{max} has no closed form.** We now need to solve $\log_{10}\mathrm{LT50}(T) = \log_{10} t_{\mathrm{ref}}$ for T per posterior draw — recovered numerically by predicting $\log_{10}\mathrm{LT50}(T)$ from the model on a fine T grid and inverting with `approx()`. The output is the same kind of posterior we already have, just produced numerically rather than algebraically.

2.0.6.1 A numerical recipe for z and CT_{max}

The simplest way to recover z and $CT_{max_{1hr}}$ from a fitted model — whether shape parameters are constant or vary with T — is to **predict survival probabilities from the model** on a fine (T, t) grid and interpolate. We can use `tidybayes::add_epred_draws()` for the prediction and base R's `approx()` for the interpolation. The same code runs whether the fit has constant shape parameters or shape parameters that vary with T.

```
# Temperatures at which we'll evaluate z. When shape parameters depend on T,
# z is itself a function of T, so we compute it at each of the original assay
# temperatures and report the full set rather than a single number.
T_eval <- temps
half_step <- 0.5  # half-width (°C) of the central finite difference window

# Build a (T, t) grid covering the assay window. The columns log10_t, T_c, and
# n must match the variables the fitted model expects on its input side - they
# are exactly the same as the columns of `design` used to fit the model.
pred_grid <- tidyr::expand_grid(
```

```
  T = seq(min(temps) - 1, max(temps) + 1, by = 0.25),
  t = 10 ^ seq(log10(0.5), log10(800), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,
    n       = 1
  )

# Now, from the fitted model, draw posterior expected survival probabilities
# at every point in the (T, t) grid. `add_epred_draws()` returns a long tibble
# with one row per (grid point × posterior draw); the column .epred is the
# predicted survival probability under that draw. ndraws keeps it manageable.
epred <- pred_grid |>
  tidybayes::add_epred_draws(fit_bin, ndraws = 500)

# For each (draw, T) we find the duration t at which predicted survival
# crosses 0.5 - that's log10(LT50)(T) for that draw. Linear interpolation
# through the grid via approx() avoids any need for root-finding.
lt50_T <- epred |>
  dplyr::group_by(.draw, T) |>
  dplyr::summarise(
    log10_lt50 = approx(.epred, log10_t, xout = 0.5)$y,
    .groups    = "drop"
  )

# Per posterior draw, compute z at each evaluation temperature in T_eval via
# a central finite difference on the lt50_T grid. For each T_at we approx()
# log10(LT50) at T_at ± half_step and divide the difference by the full step
# (2 * half_step, in °C) to get the slope; z = -1 / slope.
z_per_draw <- purrr::map_dfr(T_eval, function(T_at) {
  lt50_T |>
    dplyr::group_by(.draw) |>
    dplyr::summarise(
      T_at = T_at,
      z    = -1 / ((approx(T, log10_lt50, xout = T_at + half_step)$y -
                    approx(T, log10_lt50, xout = T_at - half_step)$y) /
                   (2 * half_step)),
      .groups = "drop"
    )
})

# Per posterior draw, CTmax_1hr is the temperature at which log10(LT50) =
# log10(60). It is a single number per draw (does not depend on T_eval).
ctmax_per_draw <- lt50_T |>
  dplyr::group_by(.draw) |>
  dplyr::summarise(
```

```
    CTmax_1hr = approx(log10_lt50, T, xout = log10(60))$y,
    .groups   = "drop"
  )

# Posterior summary of z at each evaluation temperature: median and 95% CrI.
z_per_draw |>
  dplyr::group_by(T_at) |>
  dplyr::summarise(
    z_med = median(z,         na.rm = TRUE),
    z_lo  = quantile(z,         0.025, na.rm = TRUE),
    z_hi  = quantile(z,         0.975, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 5 x 4
   T_at z_med  z_lo  z_hi
  <dbl> <dbl> <dbl> <dbl>
1    30  6.67  6.56  6.79
2    32  6.67  6.56  6.79
3    34  6.67  6.56  6.79
4    36  6.67  6.56  6.79
5    38  6.67  6.56  6.79
```

```
# Posterior summary of CTmax_1hr.
ctmax_per_draw |>
  dplyr::summarise(
    CTmax_1hr_med = median(CTmax_1hr, na.rm = TRUE),
    CTmax_1hr_lo  = quantile(CTmax_1hr, 0.025, na.rm = TRUE),
    CTmax_1hr_hi  = quantile(CTmax_1hr, 0.975, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 3
  CTmax_1hr_med CTmax_1hr_lo CTmax_1hr_hi
          <dbl>        <dbl>        <dbl>
1          32.1         32.1         32.2
```

The posterior medians of z at each of the five assay temperatures are essentially identical to one another, and the common value matches the closed-form posterior median of z from Section 2.0.4 — exactly as expected for a fit with linear `mid(T)` and constant shape parameters, where z does not depend on temperature. This flatness is the sanity check; if the same recipe is applied to a fit that lets the shape parameters vary with T (e.g. `k ~ T_c`), the same output will show z varying across the assay range, as it should. $CT_{max_{1hr}}$ likewise reproduces the closed-form posterior in `draws_bin_d` from Section 2.0.4.

2.0.6.2 Demonstrating changes with temperature-varying shape parameters

To make the points above concrete, we re-simulate beta-binomial data on the same factorial design as the simple case, but now both the dose-response steepness k and the upper asymptote u vary linearly with assay temperature. We then fit the joint Bayesian 4PL with the corresponding `bf()` formula and apply the same predict-and-interpolate recipe.

```
# (Re-defined here so this chunk can be run independently of the earlier
# simulation. Same values as the original `sim-design` chunk in @sec-sim-step2.)
temps     <- c(30, 32, 34, 36, 38)             # assay temperatures (°C)
durations <- c(1, 5, 15, 45, 135, 405)         # exposure durations (min)
n_rep     <- 30
n_per_rep <- 30
phi       <- 5                                  # beta-binomial dispersion

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - 34,                           # mean-centred T (T_bar = 34)
    n       = n_per_rep
  )

# Truth is defined on a constrained raw scale. This keeps ell in [0, 0.49],
# u in [0.51, 1], and k positive at every assay temperature. Those constraints
# avoid the label-switching and invalid-curve geometry that make identity-scale
# 4PL fits mix poorly when u and k vary with temperature.
true_params_ext <- list(
  ell_raw  = qlogis(0.05 / 0.49),          # ell = 0.05
  u_raw_0  = qlogis((0.92 - 0.51) / 0.49), # u = 0.92 at T_bar
  u_raw_T  = -0.20,                        # u declines as T rises
  log_k_0  = log(8),                       # k = 8 at T_bar
  log_k_T  = 0.10,                         # k rises as T rises
  m_beta0  = 1.5,
  m_beta1  = -0.15,
  T_bar    = 34
)

# Build per-cell true survival probabilities and draw beta-binomial counts.
# (mutate keeps all columns of `design`; mid and p_true from the original
# sim are overwritten in place with the new T-varying values.)
sim_bb_ext <- design |>
  dplyr::mutate(
    ell    = 0.49 * plogis(true_params_ext$ell_raw),
    u      = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
```

```
                                  true_params_ext$u_raw_T * T_c),
  k      = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
  mid    = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
  p_true = ell +
           (u - ell) /
           (1 + exp(k * (log10_t - mid))),
  p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
  y      = rbinom(dplyr::n(), size = n, prob = p_draw)
)
```

Lets have a look at the data along with truth (Figure S5):

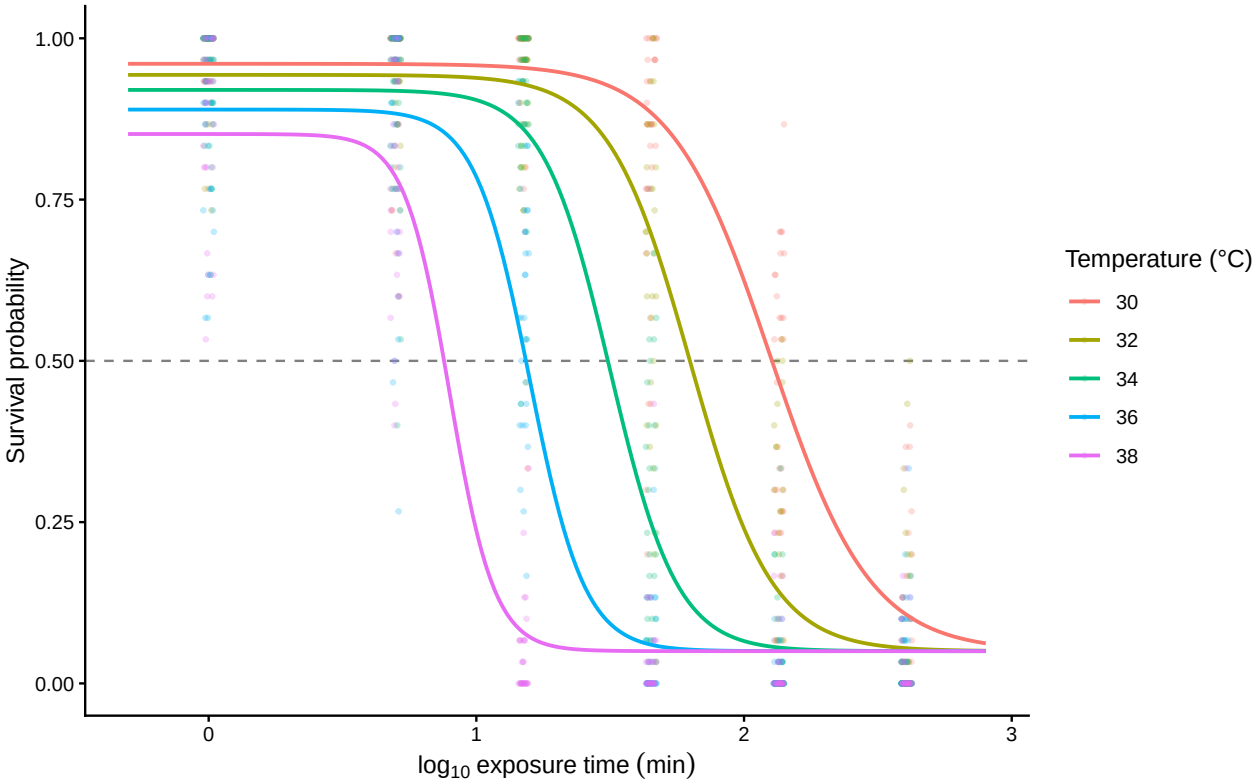

Figure S5: True 4PL dose-response surface (solid lines) for the extended simulation, where k rises and u falls linearly with assay temperature, overlaid on the simulated beta-binomial counts (points). One coloured curve and a corresponding cloud of points per assay temperature; the horizontal dashed line marks 50% survival.

Now that we have the simulated data, let's fit the model. The important change from the simple-model fit is that the four 4PL parameters are no longer fit directly on the identity scale — each one needs a different transformation because each has a different constraint:

Parameter	Natural-scale constraint	Transform used
`ell` (lower asymptote)	$[0, 0.49]$	`0.49 * inv_logit(ellraw)` — squashes raw real to $(0, 0.49)$
`u` (upper asymptote)	$[0.51, 1]$	`0.51 + 0.49 * inv_logit(uraw)` — squashes raw real to $(0.51, 1)$
`k` (slope)	> 0	`exp(logk)` — squashes raw real to $(0, \infty)$
`mid` (midpoint)	unconstrained	identity (no transform)

The asymptotes use the **disjoint** intervals $[0, 0.49]$ and $[0.51, 1]$ (rather than each covering all of $[0, 1]$) to prevent MCMC sampling issues where upper and lower asymptotes flip during MCMC sampling which can cause convergence issues — to help here we constrain them and estimate on the logit scale. The 0.02 gap at $p = 0.5$ guarantees $u > \ell$ always, so the chains can't oscillate — which matters here because u and ℓ can vary across temperatures. The cost is hard-bounding the asymptotes a few percentage points away from 0.5 — fine for survival data where genuine LT_{50} assays don't sit at the boundary anyway. We also constrain `k` to be positive, but on a log scale via `exp(logk)`. Because u, ℓ and `k` have different transformations and meanings, we apply each transformation per parameter inside the formula before they are combined — as opposed to a single link function on the linear predictor as is typical in GLMMs.

We just need to remember in our fits that these values are transformed so we need to back-transform to make sense of the coefficients.

```
# Disable Quarto chunk caching here so brms's own `file = ...` /
# `file_refit = "on_change"` mechanism is the sole source of truth for whether
# to refit. Without this, the project default `cache: true` can hold an old
# fit object even after the formula or priors change.

# Same 4PL likelihood as before, but with constrained transformations:
#   ell = 0.49 * inv_logit(ellraw)
#   u   = 0.51 + 0.49 * inv_logit(uraw)
#   k   = exp(logk)
# This guarantees ell < 0.5, u > 0.5, u > ell, and k > 0.
bform_ext <- bf(
  y | trials(n) ~
    0.49 * inv_logit(ellraw) +
      ((0.51 + 0.49 * inv_logit(uraw)) -
         0.49 * inv_logit(ellraw)) /
      (1 + exp(exp(logk) * (log10_t - mid))),
  ellraw ~ 1,
  uraw   ~ T_c,
  logk   ~ T_c,
  mid ~ T_c,
  nl = TRUE,
  family = beta_binomial(link = "identity")
)
```

```
priors_ext <- c(
  brms::prior_string(
    sprintf("normal(%f, 0.50)", true_params_ext$ell_raw),
    nlpar = "ellraw", coef = "Intercept"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.50)", true_params_ext$u_raw_0),
    nlpar = "uraw", coef = "Intercept"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.10)", true_params_ext$u_raw_T),
    nlpar = "uraw", coef = "T_c"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.35)", true_params_ext$log_k_0),
    nlpar = "logk", coef = "Intercept"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.05)", true_params_ext$log_k_T),
    nlpar = "logk", coef = "T_c"
  ),
  prior(normal(1.5, 0.40), nlpar = "mid", coef = "Intercept"),
  prior(normal(-0.15, 0.04), nlpar = "mid", coef = "T_c")
)

fit_bb_ext <- brm(
  bform_ext, data = sim_bb_ext, prior = priors_ext,
  chains = 4, iter = 2000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.95, max_treedepth = 12),
  file = file.path(models_dir, "sim_4pl_ext_betabinomial_reparam"),
  file_refit = "on_change"
)
```

The printed model summary is now mainly a convergence check because the coefficients live on raw transformed scales. We still want Rhat near 1 and reasonable effective sample sizes, but recovery is easier to inspect after transforming the posterior draws back to natural 4PL parameters.

```
# Force this chunk to re-run on every render so the printed summary always
# reflects the current `fit_bb_ext` object (the project default `cache: true`
# would otherwise hold onto a stale snapshot when the model is refit upstream).
fit_bb_ext
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: y | trials(n) ~ 0.49 * inv_logit(ellraw) + ((0.51 + 0.49 * inv_logit(uraw)) - 0.49 * 
         ellraw ~ 1
```

```
         uraw ~ T_c
         logk ~ T_c
         mid ~ T_c
   Data: sim_bb_ext (Number of observations: 900)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
ellraw_Intercept    -2.31      0.11    -2.54    -2.09 1.00     3056     3207
uraw_Intercept       1.56      0.10     1.37     1.75 1.00     2753     2571
uraw_T_c            -0.22      0.03    -0.28    -0.15 1.00     3307     2825
logk_Intercept       2.06      0.07     1.93     2.21 1.00     2700     2474
logk_T_c             0.15      0.02     0.11     0.19 1.00     3038     2790
mid_Intercept        1.50      0.01     1.48     1.53 1.00     3973     3031
mid_T_c             -0.14      0.01    -0.15    -0.13 1.00     4236     3099

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     4.75      0.36     4.08     5.49 1.00     3381     2977

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

Now we can check the changes in z and $CT_{max_{1hr}}$ from having temperature impact on multiple parameters of the 4PL using our workflow in the previous section.

```r
# Predict-and-interpolate recipe applied to the extended fit. The code is
# identical to @sec-joint-extend-recipe - only the fit changes.
pred_grid_ext <- tidyr::expand_grid(
  T = seq(min(temps) - 1, max(temps) + 1, by = 0.25),
  t = 10 ^ seq(log10(0.5), log10(800), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    n       = 1
  )

epred_ext <- pred_grid_ext |>
  tidybayes::add_epred_draws(fit_bb_ext, ndraws = 500)

lt50_T_ext <- epred_ext |>
  dplyr::group_by(.draw, T) |>
  dplyr::summarise(
    log10_lt50 = approx(.epred, log10_t, xout = 0.5)$y,
```

Table S8: Natural-scale parameter recovery from the constrained temperature-varying 4PL. The posterior is transformed from the raw `ellraw`, `uraw`, and `logk` coefficients back to the biologically interpretable asymptotes, steepness, and midpoint at each assay temperature; ϕ is the beta-binomial precision parameter used to generate the overdispersed counts.

Parameter	Temperature (°C)	Truth	Posterior median	Lower	Upper
ell	30	0.05	0.0443	0.0359	0.0539
k	30	5.363	4.3952	3.7429	5.1646
mid	30	2.1	2.0751	2.0288	2.1233
u	30	0.961	0.9603	0.9466	0.9716
ell	32	0.05	0.0443	0.0359	0.0539
k	32	6.55	5.8805	5.2081	6.673
mid	32	1.8	1.7893	1.7585	1.8221
u	32	0.943	0.9412	0.9282	0.953
ell	34	0.05	0.0443	0.0359	0.0539
k	34	8	7.8697	6.8926	9.1089
mid	34	1.5	1.5043	1.4804	1.5293
u	34	0.92	0.9148	0.901	0.9272
ell	36	0.05	0.0443	0.0359	0.0539
k	36	9.771	10.5314	8.8045	12.9561
mid	36	1.2	1.2187	1.1896	1.2488
u	36	0.89	0.8793	0.859	0.898
ell	38	0.05	0.0443	0.0359	0.0539
k	38	11.935	14.0821	10.9943	18.8182
mid	38	0.9	0.9331	0.8903	0.978
u	38	0.852	0.8355	0.8005	0.8673
phi	all	5	4.7371	4.0814	5.4931

```
    .groups    = "drop"
  )

z_per_draw_ext <- purrr::map_dfr(T_eval, function(T_at) {
  lt50_T_ext |>
    dplyr::group_by(.draw) |>
    dplyr::summarise(
      T_at = T_at,
      z    = -1 / ((approx(T, log10_lt50, xout = T_at + half_step)$y -
                    approx(T, log10_lt50, xout = T_at - half_step)$y) /
                   (2 * half_step)),
      .groups = "drop"
    )
})

ctmax_per_draw_ext <- lt50_T_ext |>
  dplyr::group_by(.draw) |>
  dplyr::summarise(
    CTmax_1hr = approx(log10_lt50, T, xout = log10(60))$y,
    .groups   = "drop"
  )

# Posterior summaries: z at each assay temperature, and CTmax_1hr.
z_summary_ext <- z_per_draw_ext |>
  dplyr::group_by(T_at) |>
  dplyr::summarise(
    z_med = median(z, na.rm = TRUE),
    z_lo  = quantile(z, 0.025, na.rm = TRUE),
    z_hi  = quantile(z, 0.975, na.rm = TRUE),
    .groups = "drop"
  )

ctmax_summary_ext <- ctmax_per_draw_ext |>
  dplyr::summarise(
    CTmax_1hr_med = median(CTmax_1hr, na.rm = TRUE),
    CTmax_1hr_lo  = quantile(CTmax_1hr, 0.025, na.rm = TRUE),
    CTmax_1hr_hi  = quantile(CTmax_1hr, 0.975, na.rm = TRUE)
  )

z_summary_ext
```

```
# A tibble: 5 x 4
   T_at z_med  z_lo  z_hi
  <dbl> <dbl> <dbl> <dbl>
1    30  6.79  6.41  7.31
2    32  6.82  6.44  7.36
3    34  6.85  6.47  7.41
```

```
4    36  6.88  6.48  7.44
5    38  6.90  6.49  7.47
```

```
ctmax_summary_ext
```

```
# A tibble: 1 x 3
  CTmax_1hr_med CTmax_1hr_lo CTmax_1hr_hi
          <dbl>        <dbl>        <dbl>
1          32.0         31.8         32.2
```

In the simple-model fit (Section 2.0.4), z was essentially the same value at every assay temperature. In this extended fit, the posterior medians of z vary across temperature (albeit slightly): a direct consequence of the temperature-dependent k and u, which bend the $\log_{10}\mathrm{LT50}(T)$ curve.

In contrast, $CT_{max_{1hr}}$ remains a single number per posterior draw, and in this simulation it actually lands very close to the simple-model value (~32 °C): the asymmetry correction is near zero at the temperature where the LT50 curve crosses 1 hour, so $CT_{max_{1hr}}$ is locally almost insensitive to the T-varying shape parameters. With more pronounced T-dependence in k and u, or with shape parameters further from the symmetric ($\ell + u \approx 1$) regime, $CT_{max_{1hr}}$ would shift more visibly.

If we now want to get an adjusted z that averages across these temperatures we can do that as follows:

```
# For each posterior draw, average the five local z(T_eval) values into
# a single adjusted z; then summarise across draws.
adjusted_z_ext <- z_per_draw_ext |>
  dplyr::group_by(.draw) |>
  dplyr::summarise(z_mean = mean(z, na.rm = TRUE), .groups = "drop")

adjusted_z_ext |>
  dplyr::summarise(
    z_med = median(z_mean,   na.rm = TRUE),
    z_lo  = quantile(z_mean, 0.025, na.rm = TRUE),
    z_hi  = quantile(z_mean, 0.975, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 3
  z_med  z_lo  z_hi
  <dbl> <dbl> <dbl>
1  6.85  6.46  7.40
```

Compared to the simple-model $z = 6.67$ °C (where shape parameters did not vary with temperature), this adjusted z is 6.85 °C with 95% credible interval [6.46, 7.4] °C — slightly lower in point estimate, with the simple-model value sitting near the upper end of the credible interval. With stronger T-dependence in k and u, the bent $\log_{10}\mathrm{LT50}(T)$ curve would have more variation across temperatures and the two summaries would diverge more.

2.0.7 Heat injury accumulation and survival under a fluctuating trace

A particularly powerful aspect of TLS models is their ability to translate dynamic temperature time-series from nature and predict how such temperatures lead to heat injury (HI) accumulation towards a threshold (e.g., LT_{50}) (Arnold *et al.* 2025; Jørgensen *et al.* 2021; Noble *et al.* 2026; Ørsted *et al.* 2022; Rezende *et al.* 2020). The joint Bayesian 4PL fit gives us not just point estimates of z and $CT_{max_{1hr}}$ but their full joint posterior, which we can push through the HI integral to obtain a posterior distribution over the predicted HI trajectory. We can also use the *full* 4PL surface to predict the cumulative survival fraction $S_{\text{pred}}(t)$ across the trajectory (Equation 9 of the manuscript). The library function `predict_heat_injury()` does both calculations in a single call and returns both trajectories with full posterior uncertainty.

2.0.7.1 A simulated temperature profile across 5 days

To mimic conditions a moderately heat-tolerant ectotherm might experience, we generate 5 days of hourly temperatures with a sinusoidal diurnal cycle, ranging from 12 °C overnight to a midday peak of 24 °C. The critical temperature T_c — below which damage is repaired as fast as it accrues and so there is no net damage increase (Jørgensen *et al.* 2021; Ørsted *et al.* 2022) — is set to 20 °C. The simulated organism has $CT_{max_{1hr}} \approx 32.1$°C and $z \approx 6.7$°C, so the daily peak at 24 °C remains 8 °C below the 1-hour critical limit — moderate, sub-lethal stress that builds cumulatively.

```
hi_trace <- tibble::tibble(
  time_h = 0:(24 * 5 - 1),
  temp   = 18 + 6 * sin(2 * pi * (0:(24 * 5 - 1) - 6) / 24)   # range 12–24 °C
)
hi_T_c <- 20    # damage threshold (°C)
```

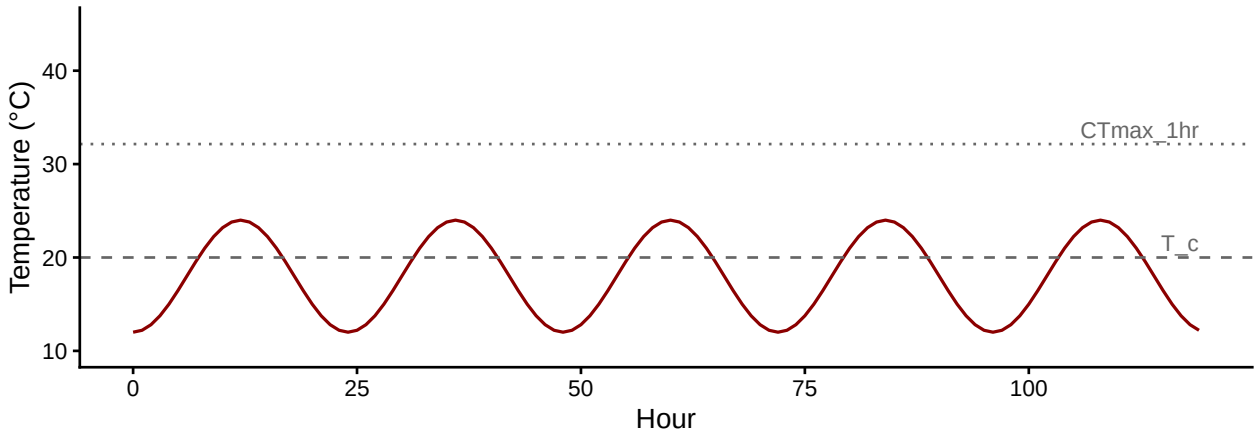

Figure S6: 5-day diurnal temperature trace. The dashed line is the damage-accumulation threshold $T_c = 20$ °C; the dotted line is the simulated organism's $CT_{max_{1hr}} \approx 32.1$°C — well above the daily peaks, so accumulation is sub-lethal and gradual.

2.0.7.2 Posterior HI and survival via `predict_heat_injury()`

`predict_heat_injury()` takes a temperature trace plus a fitted workflow, propagates the full posterior of $(\ell, u, k, \beta_0, \beta_1)$ through the HI integral, and returns the median and 95% credible band of cumulative HI and predicted survival at every time point. We supply `T_c` to enforce zero damage accumulation below the threshold (matching Equation 7 of the manuscript). The model time units here are minutes (because `standardize_data()` was called with `duration_unit = "minutes"`), so the trace's `time_h` is in hours and damage rates are computed accordingly internally.

```
# Convert hourly trace to minutes to match the model's time units.
hi_trace_min <- dplyr::mutate(hi_trace, time_h = time_h * 60)

hi <- predict_heat_injury(
  trace        = hi_trace_min,
  workflow     = wf_bb,
  target_surv  = 0.5,
  T_c          = hi_T_c,
  ndraws       = 500
)
```

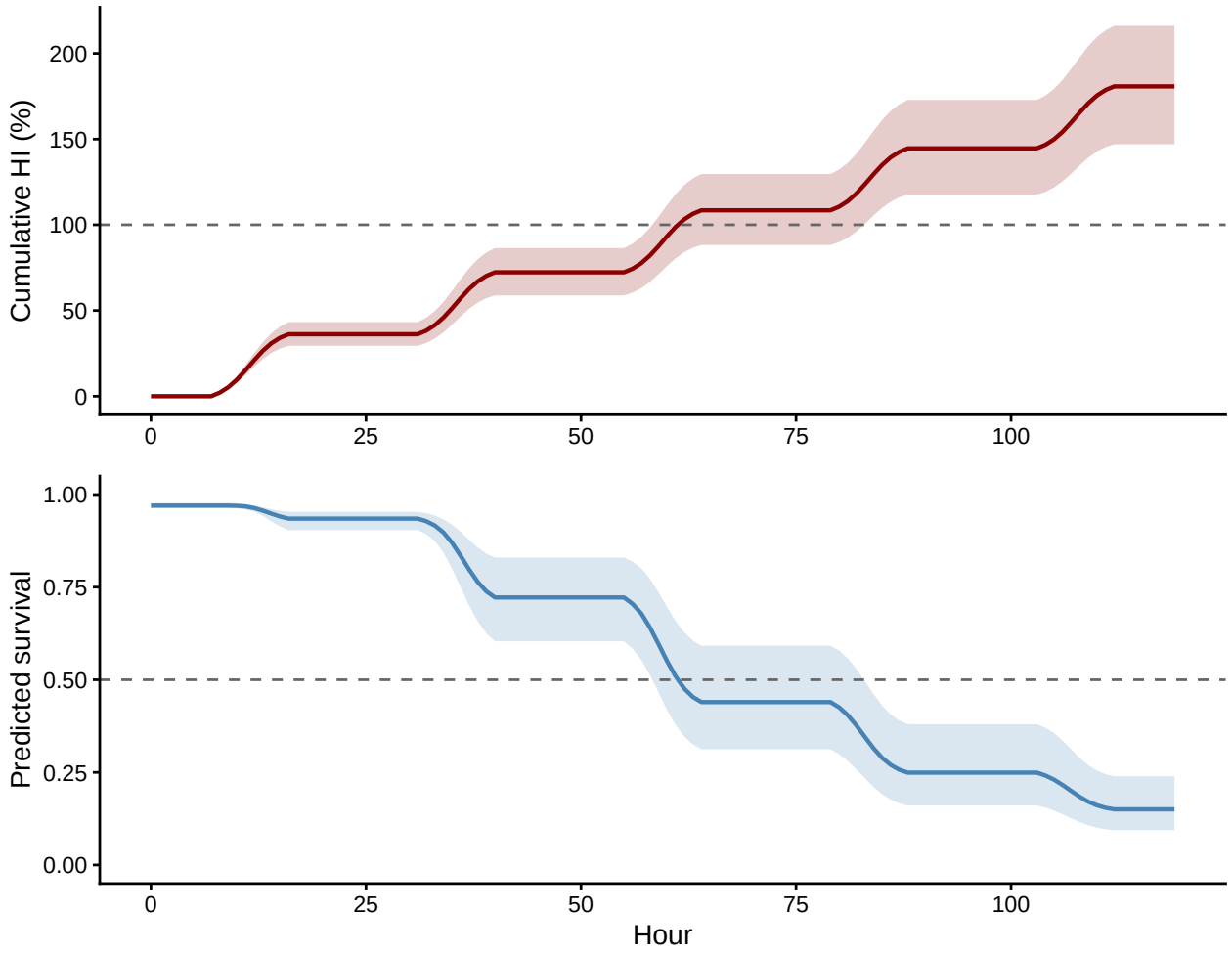

Figure S7: Posterior heat-injury accumulation (top, red) and predicted survival fraction (bottom, blue) under the 5-day field trace. Solid lines are posterior medians across 500 draws from `wf_bb`; shaded bands are 95% credible intervals. The dashed lines mark the LT50 threshold (HI = 100%) and 50% survival respectively. The two trajectories tell the same story: $S_{\text{pred}}(t)$ falls through 0.5 at the same time HI(t) crosses 100%.

By the end of day 5 the posterior median HI is 181% (95% CI: 147% to 216%), and median predicted survival is 0.15 (95% CI: 0.09 to 0.24).

2.0.8 Validation: recovering known planted doses

Before trusting `predict_heat_injury()` on real field data, we validate it against analytical truth on three reference traces with known dosing structure. `make_temperature_scenarios()` builds the traces; `planted_dose_from_trace()` implements the classical HI integral (Equation 7) directly given known `z` and `CTmax_1hr`. Posterior medians from `predict_heat_injury()` should sit near the analytical truth, and the 95% credible intervals should cover it.

```
# Three traces: flat baseline (zero HI), single hourly spike, three spikes.
val_scens <- make_temperature_scenarios(
```

```
  baseline    = 20,
  spike_temp  = 28,
  n_hours     = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72)
)
val_T_c <- 24

# Analytical truth, given the simulation parameters (true_z, true_CTmax).
planted <- purrr::map(val_scens, planted_dose_from_trace,
                      z = true_z, CTmax_1hr = true_CTmax, T_c = val_T_c)

# Posterior predictions from the fitted beta-binomial workflow. The model
# expects minutes; convert each trace's time axis.
predicted <- purrr::map(val_scens, function(sc) {
  predict_heat_injury(
    trace       = dplyr::mutate(sc, time_h = time_h * 60),
    workflow    = wf_bb,
    target_surv = 0.5,
    T_c         = val_T_c,
    ndraws      = 300
  )
})

# Final HI per scenario: analytical truth vs posterior median + CrI.
val_table <- tibble::tibble(
  Scenario     = c("Flat", "Single spike", "Multi spike"),
  `Planted HI (%)` = c(tail(planted$flat$hi_cumulative,         1),
                       tail(planted$single_spike$hi_cumulative, 1),
                       tail(planted$multi_spike$hi_cumulative,  1)),
  `Posterior median (%)` = c(tail(predicted$flat$summary$hi_median,         1),
                             tail(predicted$single_spike$summary$hi_median, 1),
                             tail(predicted$multi_spike$summary$hi_median,  1)),
  `Lower (%)` = c(tail(predicted$flat$summary$hi_lower,         1),
                  tail(predicted$single_spike$summary$hi_lower, 1),
                  tail(predicted$multi_spike$summary$hi_lower,  1)),
  `Upper (%)` = c(tail(predicted$flat$summary$hi_upper,         1),
                  tail(predicted$single_spike$summary$hi_upper, 1),
                  tail(predicted$multi_spike$summary$hi_upper,  1))
)
val_table$`Truth in CrI?` <- with(val_table,
                                  `Lower (%)` <= `Planted HI (%)` &
                                  `Planted HI (%)` <= `Upper (%)`)
tinytable::tt(val_table, digits = 2, escape = TRUE)
```

The flat trace stays below T_c for its entire duration and so accrues zero HI, matching the analytical truth exactly. The single- and multi-spike traces deliver known doses analytically;

Scenario	Planted HI (%)	Posterior median (%)	Lower (%)	Upper (%)	Truth in CrI?
Flat	0	0	0	0	TRUE
Single spike	24	23	21	26	TRUE
Multi spike	72	70	63	77	TRUE

Table S9: Summary of the shrimp lethal-TDT dataset after standardisation. Temperature centre is the grand mean used by the model for `temp_c`.

n trials	Temperature range (°C)	Duration range (hr)	Median individuals/trial	Temperature centre (°C)
148	30–33	0.08–6	10	32

`predict_heat_injury()` recovers each with the truth lying inside its credible interval. This sanity-check pattern — three traces with known doses, plus the recovery check — is what each new dataset's heat-injury prediction should pass before being trusted.

3 Case Study 1: Shrimp data

We now apply the full workflow described above — `standardize_data()` → `fit_4pl()` → `extract_tdt()` → `predict_heat_injury()` — to lethal-TDT data from brown shrimp (*Crangon crangon*). The data are 148 replicate trials spanning 30–33 °C and 5 min to 6 hours of exposure, with each replicate exposing a tank of individuals to a fixed temperature × duration combination on a specific date. The two grouping factors `Date` and `Tank` enter the model as random intercepts on `mid`, allowing day-to-day batch variation and tank-level cohort variation to be partitioned out of the dose-response surface.

3.1 Data and standardisation

```
shrimp_raw <- readxl::read_excel(
  here::here("data", "Data_Shrimp_TDT_CTmax_experiment.xlsx"),
  sheet = "LETHAL_TDT"
)

shrimp_std <- standardize_data(
  data           = shrimp_raw,
  temp           = "Temperature_assay",
  duration       = "Duration_exposure_hours",
  n_total        = "N_individuals_after_trial",
  mortality      = "Mortality_after_trial",
  random_effects = c("Date", "Tank"),
  duration_unit  = "hours"
)
```

Table S10: Sampling diagnostics for the shrimp 4PL fit. Healthy targets: Rhat < 1.01, ESS > 400, zero divergences.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	rhat_pass	ess_pass	divergence_
1	1468	2374	0	2	TRUE	TRUE	TRUE

Table S11: Posterior medians and 95% credible intervals for the shrimp 4PL fit, transformed back to the natural scale. `low` and `up` are the bottom and top asymptotes of the survival curve at the grand-mean temperature; `k` is the slope on $\log_{10}$ time; `mid` intercept is $\log_{10}$ LT50 in hours at the grand-mean temperature; `mid temp_c slope` is β_1, from which $z = -1/\beta_1$.

parameter	median	lower	upper
low (lower asymptote)	0.00692	0.00228	0.0222
up (upper asymptote)	0.95174	0.89351	0.9928
k (slope)	5.82927	4.68871	7.4851
mid intercept (at T_bar)	0.05697	-0.30812	0.4094
mid temp_c slope	-0.36949	-0.42841	-0.3109
z (°C)	2.70643	2.33424	3.2163

3.2 Fitting the joint Bayesian 4PL

We fit the default `fit_4pl()` model — beta-binomial likelihood, identity link, the disjoint-bounds asymptote reparameterisation, and `temp_c` slopes on all four 4PL sub-parameters — with random intercepts on `mid` for `Date` and `Tank`. The fit takes a few minutes on first render; subsequent renders reload it from disk because `file_refit = "on_change"`.

```
wf_shrimp <- fit_4pl(
  shrimp_std,
  random_effects = c("Date", "Tank"),
  chains  = 4, iter = 3000, warmup = 1500, cores = 4, seed = 123,
  control = list(adapt_delta = 0.99, max_treedepth = 12),
  file    = file.path(models_dir, "fit_shrimp_lethal_4pl")
)
```

3.2.1 Sampling diagnostics

3.2.2 Posterior parameters on the natural scale

3.3 Survival curves with observed overlay

We predict survival probability across the observed temperature × duration grid and overlay the observed proportions per replicate. The fitted curves should track the observed clouds at every temperature.

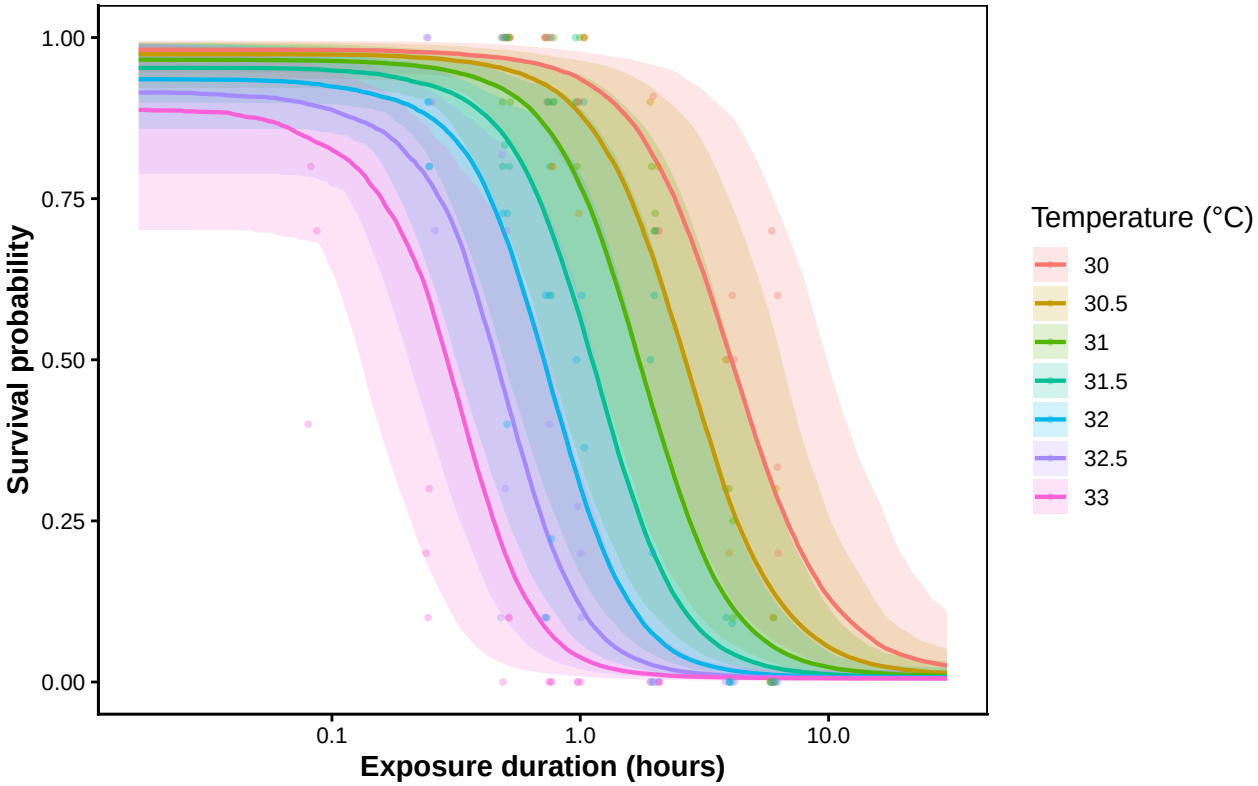

Figure S8: Posterior survival curves for brown shrimp at each assayed temperature, with 95% credible bands. Coloured points are observed per-replicate survival proportions. Duration on the log scale; population-level prediction (random effects marginalised).

3.4 TDT landscape

A dense 2-D view of survival across the entire (temperature, duration) plane shows the dose-response surface as a heatmap with overlaid contours at 25, 50, and 75% survival.

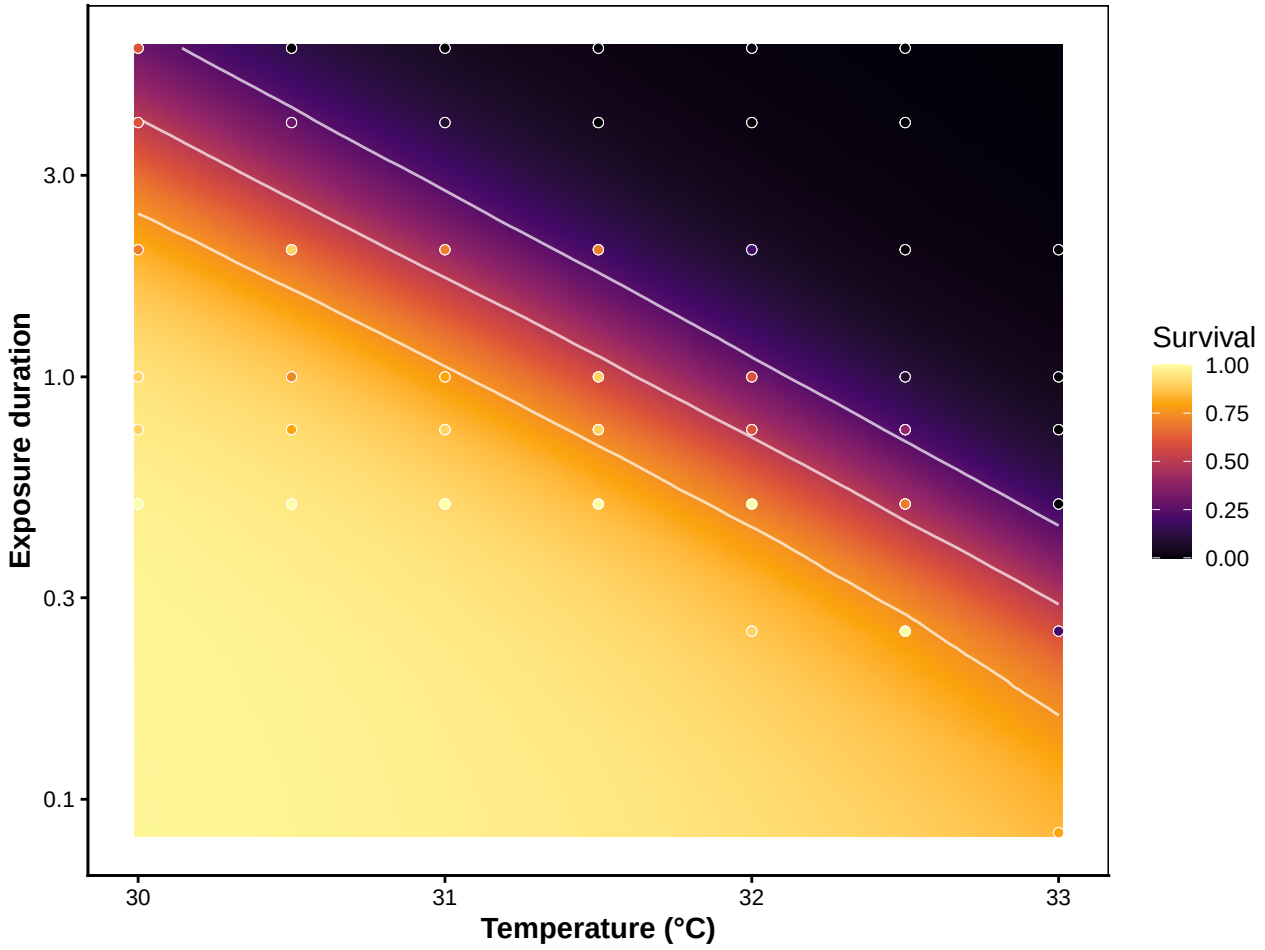

Figure S9: TDT landscape for brown shrimp: posterior median predicted survival across a 120 × 120 grid of assay temperatures × durations. White contours mark 25%, 50%, and 75% survival isolines. Overlaid points are raw observed proportions.

3.5 Classical TDT quantities: z, $CT_{max_{1hr}}$, T_{crit}

`extract_tdt()` bundles z (from per-draw log-linear regression of the LT50 curve), $CT_{max_{1hr}}$ (4PL inversion at the 60-minute reference for 50% survival), and T_{crit} (the rate-multiplier damage-threshold, integrated over $r^* \in [0.1, 1]$ % HI per hour — see Equation S3) in a single call. Every output inherits the model's posterior, so all three quantities come with calibrated credible intervals.

```
et_shrimp <- extract_tdt(
  wf_shrimp,
  t_ref         = 60,            # CTmax defined at 1-hour exposure
  TC_rate_range = c(0.1, 1),     # T_crit integrated over r* in % HI/hr
  ndraws        = 500
)
```

Table S12: Classical TDT summaries for brown shrimp, with full posterior uncertainty. z comes from a per-draw log-linear regression of $\log_{10}$ LT50 on temperature. $CT_{max_{1hr}}$ is the temperature at 50% survival after 1-hour exposure (4PL inversion per draw). T_{crit} is the temperature at which the TDT damage rate falls below r^* % HI per hour, with r^* sampled uniformly on $\log_{10}$ across $[0.1, 1]$ — the pooled posterior carries both parameter uncertainty and operational uncertainty in r^*.

Quantity	Median	Lower	Upper
z (°C per decade of time)	2.6	2.3	3
CTmax_1hr (°C)	31.6	30.7	33
T_crit (°C, r* in [0.1, 1] per hour)	25.1	22.9	27

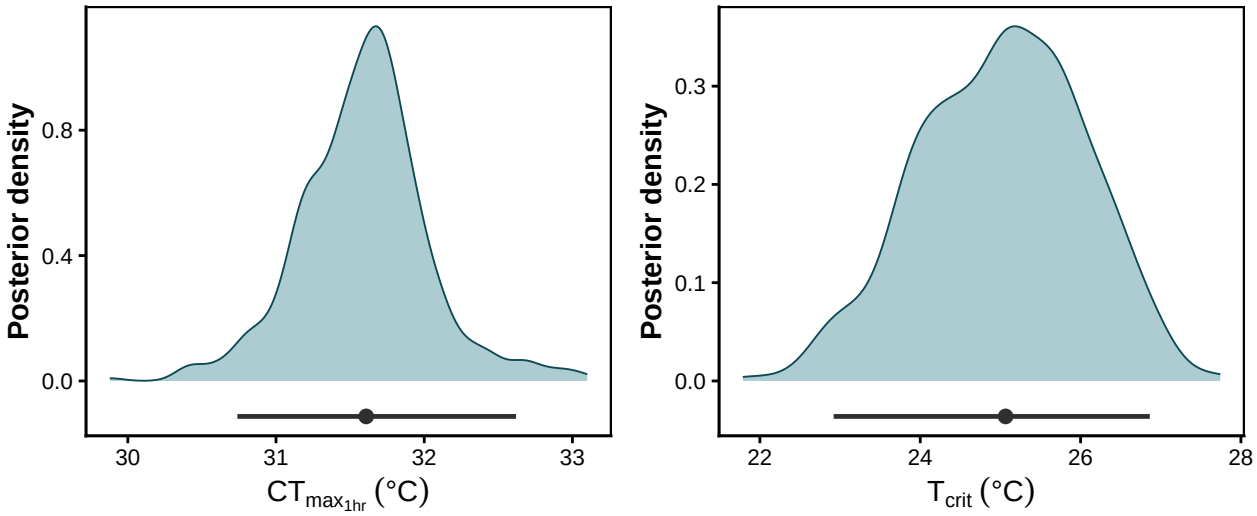

Figure S10: Posterior densities of $CT_{max_{1hr}}$ (left) and T_{crit} (right) for brown shrimp. The horizontal bar below each density is the 95% credible interval; the point marks the posterior median. The 5–6 °C gap between the two quantities represents the sub-lethal window between the onset of detectable injury and the LT50 limit.

3.6 Heat injury under three reference traces

We now project the fitted shrimp model under the same three-scenario harness used in the simulation validation above (§Validation: recovering known planted doses). The traces are calibrated to shrimp ecology and the posterior of the fitted model: 17 °C is the natural holding temperature, 30 °C is a heat-stress spike that sits between T_{crit} and $CT_{max_{1hr}}$, and the damage threshold passed to `predict_heat_injury()` is the posterior median of T_{crit} from `extract_tdt()` — model-derived rather than hand-picked.

```
# Damage threshold = posterior median of T_crit from the fit. Model-derived.
shrimp_T_c <- et_shrimp$T_crit$summary$temp_median

shrimp_scens <- make_temperature_scenarios(
```

```
  baseline   = 17,
  spike_temp = 30,    # between T_crit and CTmax_1hr - clearly stressful
  n_hours    = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72)
)

# Predict HI + survival under each scenario, without repair.
shrimp_hi <- purrr::map(shrimp_scens, function(sc) {
  predict_heat_injury(
    trace       = sc,
    workflow    = wf_shrimp,
    target_surv = 0.5,
    T_c         = shrimp_T_c,
    ndraws      = 500
  )
})

# Sharpe-Schoolfield repair parameters tuned to shrimp: optimum ~17 °C, modest
# repair capacity. r_ref is in (doses per hour) because the model time unit
# for the shrimp fit is hours.
shrimp_repair_pars <- list(
  TA   = 14065,
  TAL  = 50000,
  TAH  = 120000,
  TL   = 10.5 + 273.15,
  TH   = 22.5 + 273.15,
  TREF = 17   + 273.15,
  r_ref = 0.005    # 0.5% LT50-dose repaired per hour at TREF
)

# Re-run with repair on the multi-spike scenario to show the offsetting effect.
shrimp_hi_repair <- predict_heat_injury(
  trace       = shrimp_scens$multi_spike,
  workflow    = wf_shrimp,
  target_surv = 0.5,
  T_c         = shrimp_T_c,
  ndraws      = 500,
  repair      = TRUE,
  repair_pars = shrimp_repair_pars
)
```

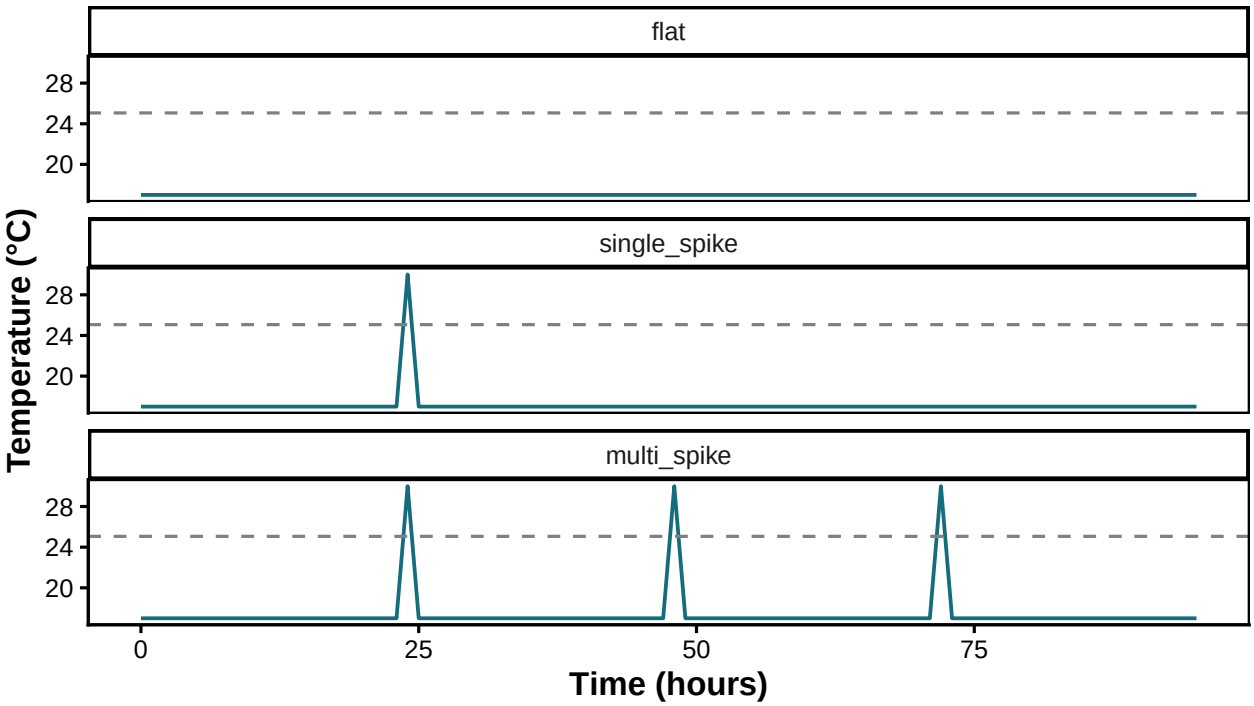

Figure S11: The three reference temperature traces used for the shrimp heat-injury prediction. Baseline 17 °C (shrimp natural holding temperature); 1-hour spikes at 28 °C; the dashed line marks the damage threshold T_c set at the lower 95% credible bound of T_crit.

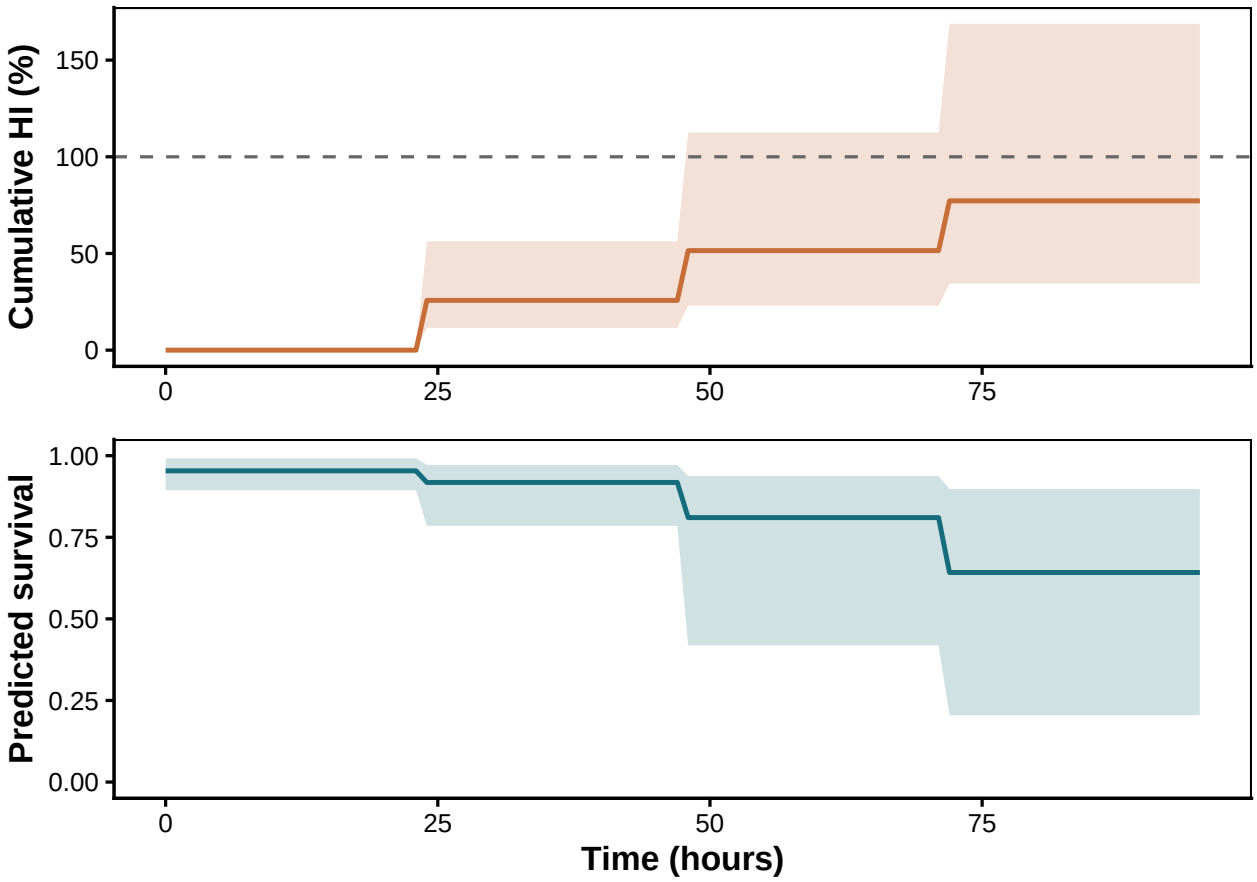

Figure S12: Cumulative heat injury (top) and predicted survival (bottom) for brown shrimp under the multi-spike trace, **no repair**. Posterior median lines and 95% credible bands.

Table S13: Final cumulative HI and predicted survival under each scenario for brown shrimp, posterior medians with 95% credible intervals.

Scenario	Repair	HI median (%)	HI lower	HI upper	Survival median	Survival lower	S
Flat	None	0.0	0.0	0.0	0.951	0.895	0.
Single spike	None	25.4	12.1	50.3	0.920	0.816	0.
Multi spike	None	77.2	34.5	168.8	0.642	0.205	0.
Multi spike	Sharpe-Schoolfield	44.6	0.0	165.3	0.799	0.224	0.

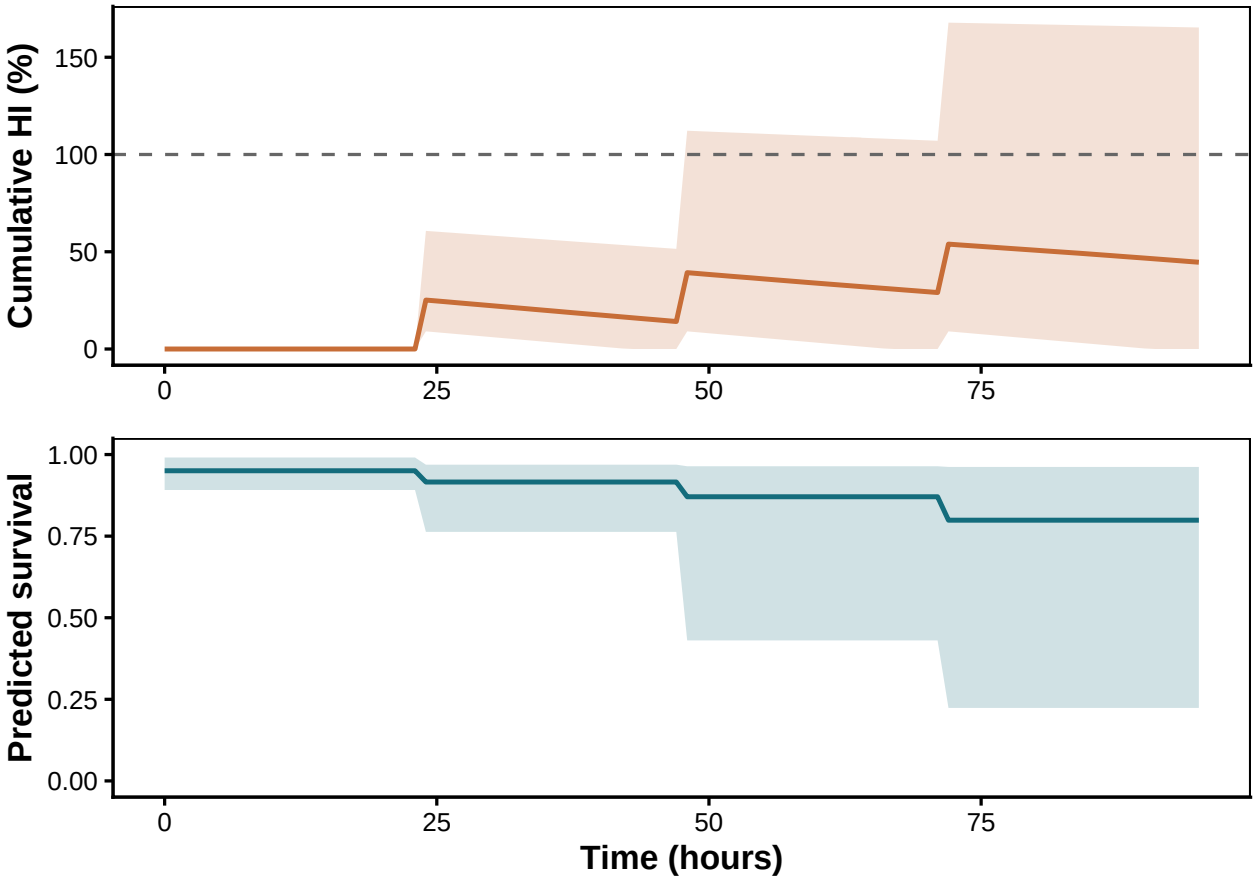

Figure S13: Same multi-spike trace as above, this time **with Sharpe-Schoolfield repair** active (`r_ref = 0.005` doses per hour at 17 °C). Repair fully offsets the spike damage at sub-T_c baseline.

The shrimp case study shows the complete loop: raw counts → fitted model → classical TDT quantities (with credible intervals on every one of them) → heat-injury predictions under fluctuating field traces (with and without repair). Every quantity reported here inherits the joint posterior of the single 4PL fit — there is no second-stage regression, no point-estimate-then-bootstrap, no separate ramping-rate experiment. The same machinery applies to any TDT-style proportion dataset by changing only `standardize_data()`'s column-name arguments.

4 References

Arnold, P.A., Noble, D.W.A., Nicotra, A.B., Kearney, M.R., Rezende, E.L., Andrew, S.C., *et al.* (2025). A framework for modelling thermal load sensitivity across life. *Global Change Biology*, 31, e70315.

Jørgensen, L.B., Malte, H. & Overgaard, J. (2021). A unifying model to estimate thermal tolerance limits in ectotherms across static, dynamic and fluctuating exposures to thermal stress. *Scientific Reports*, 11, 12840.

Kingsolver, J.G. & Umbanhowar, J. (2018). The analysis and interpretation of critical temperatures. *Journal of Experimental Biology*, 221, jeb167858.

Noble, D.W.A., Mayfield, M.M., Hoffmann, A.A., Chen, Z.-H., Lade, S.J., Bai, X., *et al.* (2026). A systems modelling approach to predict biological responses to extreme heat. *Trends in Ecology & Evolution*, 41, 451–464.

Ørsted, M., Jørgensen, L.B. & Overgaard, J. (2022). Finding the right thermal limit: A framework to reconcile ecological, physiological and methodological aspects of CTmax in ectotherms. *Journal of Experimental Biology*, 225, jeb244514.

Rezende, E.L., Bozinovic, F., Szilágyi, A. & Santos, M. (2020). Predicting temperature mortality and selection in natural Drosophila populations. *Science*, 369, 1242–1245.

Rezende, E.L., Castañeda, L.E. & Santos, M. (2014). Tolerance landscapes in thermal ecology. *Functional Ecology*, 28, 799–809.

5 Session Information

R version 4.5.3 (2026-03-11)

Platform: aarch64-apple-darwin20

locale: C.UTF-8||C.UTF-8||C.UTF-8||C||C.UTF-8||C.UTF-8

attached base packages: *stats*, *graphics*, *grDevices*, *utils*, *datasets*, *methods* and *base*

other attached packages: *pander(v.0.6.6)*, *tinytable(v.0.16.0)*, *tidybayes(v.3.0.7)*, *posterior(v.1.6.1)*, *brms(v.2.23.0)*, *Rcpp(v.1.1.1-1.1)*, *ggplot2(v.4.0.3)*, *purrr(v.1.2.2)*, *tibble(v.3.3.1)*, *tidyr(v.1.3.2)*, *dplyr(v.1.2.1)* and *here(v.1.0.1)*

loaded via a namespace (and not attached): *svUnit(v.1.0.8)*, *tidyselect(v.1.2.1)*, *farver(v.2.1.2)*, *loo(v.2.9.0)*, *S7(v.0.2.2)*, *fastmap(v.1.2.0)*, *TH.data(v.1.1-3)*, *tensorA(v.0.36.2.1)*, *pacman(v.0.5.1)*, *digest(v.0.6.39)*, *estimability(v.1.5.1)*, *lifecycle(v.1.0.5)*, *StanHeaders(v.2.32.10)*, *processx(v.3.9.0)*, *survival(v.3.8-6)*, *magrittr(v.2.0.5)*, *compiler(v.4.5.3)*, *rlang(v.1.2.0)*, *tools(v.4.5.3)*, *yaml(v.2.3.12)*, *knitr(v.1.51)*, *labeling(v.0.4.3)*, *bridgesampling(v.1.2-1)*, *pkgbuild(v.1.4.8)*, *curl(v.7.1.0)*, *plyr(v.1.8.9)*, *RColorBrewer(v.1.1-3)*, *multcomp(v.1.4-28)*, *abind(v.1.4-8)*, *withr(v.3.0.2)*, *grid(v.4.5.3)*, *stats4(v.4.5.3)*, *colorspace(v.2.1-1)*, *xtable(v.1.8-8)*, *inline(v.0.3.21)*, *emmeans(v.2.0.3)*, *scales(v.1.4.0)*, *MASS(v.7.3-65)*, *tinytex(v.0.59)*, *cli(v.3.6.6)*, *mvtnorm(v.1.3-7)*, *rmarkdown(v.2.31)*, *generics(v.0.1.4)*, *otel(v.0.2.0)*, *RcppParallel(v.5.1.11-2)*, *reshape2(v.1.4.5)*, *rstan(v.2.32.7)*, *stringr(v.1.6.0)*, *splines(v.4.5.3)*, *bayesplot(v.1.15.0)*, *parallel(v.4.5.3)*, *matrixStats(v.1.5.0)*, *vctrs(v.0.7.3)*, *V8(v.6.0.4)*, *Matrix(v.1.7-4)*, *sandwich(v.3.1-1)*, *jsonlite(v.2.0.0)*, *callr(v.3.7.6)*, *litedown(v.0.7)*, *arrayhelpers(v.1.1-0)*, *ggdist(v.3.3.3)*, *glue(v.1.8.1)*, *codetools(v.0.2-20)*, *distributional(v.0.6.0)*, *stringi(v.1.8.7)*, *gtable(v.0.3.6)*, *QuickJSR(v.1.9.0)*, *pillar(v.1.11.1)*,

htmltools(v.0.5.9), *Brobdingnag(v.1.2-9)*, *R6(v.2.6.1)*, *rprojroot(v.2.1.1)*, *evaluate(v.1.0.5)*, *lattice(v.0.22-9)*, *backports(v.1.5.1)*, *rstantools(v.2.6.0)*, *coda(v.0.19-4.1)*, *gridExtra(v.2.3)*, *nlme(v.3.1-168)*, *checkmate(v.2.3.4)*, *mgcv(v.1.9-4)*, *xfun(v.0.57)*, *zoo(v.1.8-14)* and *pkgconfig(v.2.0.3)*

:::